AI-Powered Full Stack Developer Program

STUDENT NOTES

Week 2 · Day 2

Semantic HTML, Accessibility, Media → CSS Basics

Theory notes 01–11: read them before homework, keep them for interviews

Momo House · TSI Canteen · Aarav · Sunita · Didi · Rajan

Stage 1, AI as Tutor: AI may explain; you type every line.

Tech Skills Institute (TSI) Nepal · Kathmandu

W2D2 Student Notes: Semantic HTML, Accessibility, Media → CSS Basics

File: notes/README.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2 · Day 2 · Layer: Frontend

Today in one sentence: first you give the skeleton's boxes real names, then you dress the skeleton with CSS.

Yesterday (Day 1) you built and repaired HTML skeletons. Today you do two things:

1. **Tidy the skeleton.** Use boxes with names (`<header>`, `<nav>`, `<main>`…), make the page work for everyone (accessibility), and add sound, video and a window into another page.
2. **Dress the skeleton.** Write CSS: rules, selectors, colours, units, fonts, the box model and display.

All the examples use the same story as class: **Momo House** (Didi's momo restaurant in Kathmandu), the **TSI Canteen**, and the students **Aarav**, **Sunita** and **Rajan**.

How to use these notes

- Read a note **after** its class block, or before the homework.
- Every code block that starts with `File:` is a **complete file**. Make a new folder, create the file with that exact name, paste nothing: **type it**. Then open it with Live Server.
- Images and media: copy the SVG files you need from `W2D2/assets/images/` into an `images/` folder next to your file (and `.wav`/`.mp4` from `W2D2/assets/media/` into a `media/` folder).
- Code marked **broken** in the text is wrong on purpose. It shows a mistake. Do not copy it into your projects.
- **Predict:** lines ask you to guess before you run. Guess first. Then check. This habit makes you a strong developer.
- "Check yourself" questions have no answers here. Test yourself, then check with the code, your notes or the HW3 sheet.

The notes

#	File	Class block	What it covers
01	01-semantic-html5.md	B1	header, nav, main, section, article, aside, footer, figure; div and span; button vs div
02	02-accessibility-and-meta.md	B2	lang, alt, link text, labels, heading order, contrast, title, meta description, viewport, favicon, Lighthouse, the Tab walk
03	03-audio-video-iframe.md	B3	audio, video, source + type, poster, autoplay + muted, fallback text, iframe + title
04	04-css-basics-and-linking.md	B0 + B4	skeleton & outfit, rule anatomy, inline / internal / external CSS, comments, the cascade, DevTools Styles
05	05-selectors.md	B5	element, .class, #id, grouping, descendant, .a.b, :hover, light specificity
06	06-colours-units-fonts.md	B6	named / hex / rgb / rgba, px / % / rem / em, font stacks, text properties
07	07-box-model.md	B7	content, padding, border, margin, shorthand, margin: 0 auto, box-sizing, border-radius
08	08-display.md	B8	block, inline, inline-block, none vs visibility: hidden, a horizontal nav
09	09-debugging-css.md	all CSS blocks	every CSS bug type + the table **"what the browser drops"**
10	10-bootstrap-teaser.md	B10	Bootstrap = IKEA for websites: container, btn, card
11	11-day2-cheatsheet.md	all	one-page summary to keep next to your keyboard

The bug visibility legend (same as class)

Tag	Meaning	Tool that shows it
V	You can **see** it on the page	your eyes
D(HH)	HTMLHint underlines it in VS Code / Problems panel (Ctrl+Shift+M)	HTMLHint
D(F12)	Only the Elements panel shows what the browser built	Chrome DevTools
D(LH)	Lighthouse → Accessibility report (works offline)	Chrome DevTools
D(W3C)	Only the W3C validator reports it	validator.w3.org (Wi-Fi)
H	Hidden on Windows, breaks on a real Linux server (file-name case)	the instructor
R	**Rule check**: no tool fires; read the code against the rule	your brain + these notes

AI rule for this week

Stage 1: AI as Tutor. AI may **explain** any concept or error. You type every line yourself. Try to debug first; ask AI only after you have tried.

---

*Remember the One Rule: **You may only ship code you can explain, line by line.***

01 · Semantic HTML5: Boxes with Names

File: notes/01-semantic-html5.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2 · Day 2 · Block B1 **Layer:** Frontend (HTML structure)

Today in one sentence: use tags whose *names* say what the box is for, so people, Google, screen readers and AI can understand your page.

The picture to keep in your head

Boxes / rooms with names: `<div class="header">` → `<header>`; Google, screen readers and AI read the names.

Think of a house. A good house plan does not say "room 1, room 2, room 3". It says **kitchen**, **bedroom**, **bathroom**. When a guest arrives, they know where to go. When a plumber comes, they go straight to the kitchen and the bathroom.

A web page is the same. A page made only of `<div>` boxes is a house with "room 1, room 2, room 3". It can *look* fine, but nobody can tell which box is the menu and which box is the main story. Semantic tags are the room names.

Analogy (house)	Real thing (HTML)
The sign over the front door	`<header>` (logo, site name)
The hallway with doors to every room	`<nav>` (the main links)
The main living room, only one per house	`<main>` (the main content of this page)
A themed corner of a room, with its own sign	`<section>` (a group of content **with a heading**)
A letter you could post to someone else and it still makes sense	`<article>` (a blog post, a news item, a product card)
A side table with extra things	`<aside>` (related links, tips, ads)
The back door with the address plate	`<footer>` (contact, copyright)
An unlabeled cardboard box	`<div>` (a block box with no meaning)
A highlighter on a few words	`<span>` (an inline box with no meaning)

Words you now own

Term	What it means (plain English)	Example
Semantic	"Has meaning." A semantic tag tells you what its content **is**, not how it looks.	`<nav>` = "these are the main links"
Landmark	A big named area of the page that screen readers can jump to.	`<header>`, `<nav>`, `<main>`, `<footer>`, `<aside>`
`<header>`	The top area of a page or of an article: logo, title, maybe the nav.	Momo House logo + name
`<nav>`	A block of **main navigation links**. Not every group of links: only the main ones.	Home · Menu · Order · About
`<main>`	The main content of this page. **Only one per page.**	The menu table on `menu.html`
`<section>`	A themed group of content. **It should start with a heading.**	"Today's specials" with its `<h2>`
`<article>`	A self-contained piece that makes sense alone.	One blog post on the TSI Blog
`<aside>`	Content beside the main story: related, but not the main thing.	"Related posts", a tip box
`<footer>`	The bottom area: contact, copyright, small links.	"© 2026 Momo House, Thamel"
`<figure>` + `<figcaption>`	An image (or a chart, or code) with its caption.	A momo photo with "Steam momo, Rs. 150"
`<div>` / `<span>`	Boxes with no meaning. `div` is a block, `span` is inline. Use them only for styling hooks.	`<div class="row">`, `<span class="price">`

Term	What it means (plain English)	Example
`<button>`	A real button: the Tab key reaches it, Enter and Space press it.	`<button>Subscribe</button>`

How it works

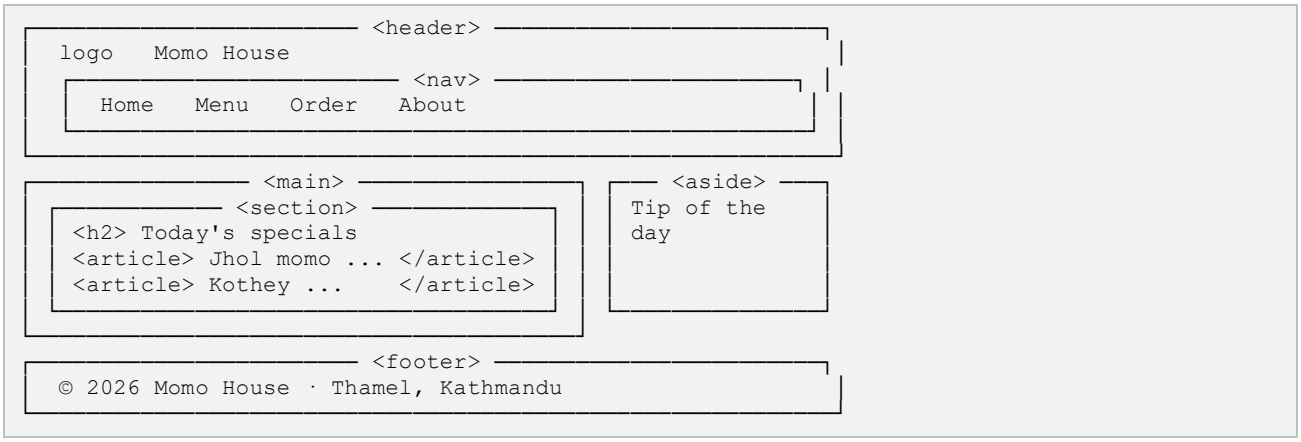
Why bother? The page looks the same

Semantic tags have almost **no look** of their own. `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>` and `<footer>` all display like a `<div>`: a block on its own line. So why change?

Because four readers use your page, and only one of them has eyes:

1. **You and your team.** `</main>` is easier to read than the fifth `</div>` in a row.
2. **Screen readers.** A blind visitor can press one key and jump to "main" or "navigation". With only divs, there is nothing to jump to.
3. **Google and other search engines.** They give more weight to the text inside `<main>` and `<article>`.
4. **AI tools.** When you paste your page into an AI tutor, named boxes help it understand the page too.

A typical page plan



Today we have no CSS layout yet (no flexbox or grid), so `<aside>` will sit **under** `<main>` on your screen, not beside it. That is fine. The *names* are what matter today.

section vs article vs div: the three questions

Ask these questions in order:

1. **Would this make sense on its own**, posted somewhere else (a post, a news item, a product card)? → `<article>`
2. **Is it a themed group with its own heading?** → `<section>`
3. **Do I only need a box to hang CSS on?** → `<div>`

One main per page

`<main>` means "the main content of **this** page". A page has one main story, so it has **one** `<main>`. Two `<main>` elements confuse screen readers. The W3C validator reports it; your eyes and HTMLHint do not. That is why it is an **R** (rule check) bug in class.

Sections need headings

A `<section>` is a themed group, and a theme has a name. So every `<section>` should start with a heading (`<h2>`, `<h3>`…). If you have a group with no heading, it is probably just a `<div>`.

nav is for the main links only

`<nav>` is the hallway. It holds the **main links** of the site. It is not for a paragraph of text, and not for every single link on the page (a link inside a sentence stays in its `<p>`).

button vs div: the Tab key test

A `<div class="btn">Subscribe</div>` can be styled to *look* like a button. But:

- the **Tab** key skips it (a keyboard user can never reach it),
- **Enter** and **Space** do nothing,
- a screen reader says "Subscribe", not "Subscribe, button".

A real `<button>` gets all of this for free. The rule: **if you click it to do something, use `<button>`. If you click it to go somewhere, use `<a href>`.**

Linked editing: rename both tags at once

The class `W2D2` folder turns on `"editor.linkedEditing": true`. Put your cursor in `div` of `<div class="header">`, delete `div` and type `header`. The closing `</div>` changes to `</header>` by itself. Then delete the `class="header"` part.

Code you built today

Step 1: the div soup (before)

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House | Div soup</title>
</head>
<body>
  <div class="header">
    <img src="images/logo-momo-house.svg" alt="Momo House logo" width="80">
    <h1>Momo House</h1>
    <div class="nav">
      <a href="index.html">Home</a> |
      <a href="index.html">Menu</a> |
      <a href="index.html">About</a>
    </div>
  </div>
  <div class="main">
    <div class="section">
      <h2>Today's specials</h2>
      <div class="article">
        <h3>Jhol momo</h3>
        <p>Steamed momo in a warm sesame and tomato soup. Rs. 200.</p>
      </div>
    </div>
  </div>
  <div class="aside">
    <p>Tip: order before 12:00 and skip the lunch queue.</p>
  </div>
  <div class="footer">
    <p>&copy; 2026 Momo House, Thamel, Kathmandu</p>
  </div>
</body>
</html>
```

- Every box is a `<div>`. The class names *try* to say what each box is, but browsers, Google and screen readers ignore class names.

Predict: open F12 → Elements. How many landmarks (header, nav, main…) does the browser know about? (Look at the tag names, not the class names.)

Step 2: the same page, with names (after)

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House | Home</title>
</head>
<body>
  <header>
    <img src="images/logo-momo-house.svg" alt="Momo House logo" width="80">
    <h1>Momo House</h1>
    <nav>
      <a href="index.html">Home</a> |
      <a href="index.html">Menu</a> |
      <a href="index.html">About</a>
    </nav>
  </header>
  <main>
    <section>
      <h2>Today's specials</h2>
      <article>
        <h3>Jhol momo</h3>
        <p>Steamed momo in a warm sesame and tomato soup. Rs. 200.</p>
      </article>
    </section>
  </main>
  <aside>
    <p>Tip: order before 12:00 and skip the lunch queue.</p>
  </aside>
  <footer>
    <p>&copy; 2026 Momo House, Thamel, Kathmandu</p>
  </footer>
</body>
</html>
```

- `<div class="header">` became `<header>` (the same for nav, main, section, article, aside, footer).
- The page **looks exactly the same**. Only the names changed. That is the point.
- `<section>` starts with its `<h2>`. `<article>` (one dish) makes sense alone.

Predict: press Ctrl+F5. Did anything on the screen move? Now look in the Elements panel. What changed?

Step 3: figure, a real button, and a link

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TSI Blog | Momo week</title>
</head>
<body>
  <header>
    <h1>TSI Blog</h1>
    <nav>
      <a href="index.html">Home</a> |
      <a href="index.html">Posts</a> |
      <a href="index.html">Contact</a>
    </nav>
  </header>
  <main>
    <article>
      <h2>Sunita tries every momo in Thamel</h2>
      <p>Sunita Gurung spent one week at Momo House. Here is what she learnt.</p>
      <figure>
        <img src="images/momo-plate.svg" alt="A plate of ten steamed momo with red chutney" width="300">
        <figcaption>Didi's steam momo, Rs. 150 a plate.</figcaption>
      </figure>
      <p>Her favourite was the jhol momo.</p>
    </article>
    <section>
      <h2>Get new posts</h2>
      <p>One email every Friday. No spam.</p>
      <button type="button">Subscribe</button>
```

```
    </section>
  </main>
  <aside>
    <h2>Related posts</h2>
    <p><a href="index.html">Aarav's first week at TSI</a></p>
  </aside>
  <footer>
    <p>&copy; 2026 TSI Blog, Dillibazar, Kathmandu</p>
  </footer>
</body>
</html>
```

- The **main blog post** is the `<article>` inside `<main>`. "Related posts" is extra, so it goes in `<aside>`.
- `<figure>` keeps the photo and its caption together.
- `<button>` is reachable with the Tab key. (It does nothing yet. JavaScript comes in session 4.)

Predict: click in the page and press **Tab** again and again. Which things get a focus ring, and in what order?

Quick reference

You have…	Use
Logo + site name at the top	`<header>`
The main menu links	`<nav>`
The main content of this page (once)	`<main>`
A themed group **with a heading**	`<section>`
A blog post, news item, product or review card	`<article>`
Related links, tips, ads	`<aside>`
Contact, copyright, small print	`<footer>`
An image with a caption	`<figure>` + `<figcaption>`
A box only for styling	`<div>` (block) / `<span>` (inline)
Something you click to **do** something	`<button>`
Something you click to **go** somewhere	`<a href="…">`

Mistakes everyone makes

You see…	Because…	Fix	Planted in
The validator says "duplicate main"	Two `<main>` elements on one page	Keep one `<main>`; make the other a `<section>`	D2-02 bug 1, D2-HW1 bug 2
A `<nav>` full of sentences	`<nav>` wraps a paragraph, not links	Put the text in `<p>`; keep `<nav>` for the main links	D2-02 bug 2
A `<section>` with no title	Sections are themed groups, and themes have names	Add an `<h2>`, or change it to a `<div>`	D2-02 bug 3, D2-HW1 bug 3
The main post is in `<aside>`, the extras are in `<main>`	The boxes are swapped	Main story → `<main>`; related box → `<aside>`	D2-02 bug 4
The Tab key skips your "button"	`<div class="btn">` is not a button	Use `<button>`	D2-02 bug 5, D2-FF5 bug 5
`<header>` inside `<footer>`	Wrong nesting of landmarks	Header at the top, footer at the bottom, not inside each other	D2-FF5 bug 1
The site menu is an `<article>`	`<article>` is for self-contained content	Use `<nav>` for the menu	D2-FF5 bug 2
"Warning:" in `<b>`	`<b>` only makes text bold; it carries no meaning	Use `<strong>` for important text	D2-FF5 bug 3

Check yourself

1. Name the seven semantic tags you met today, and say in one line what each one is for.
2. The page looks the same with divs and with semantic tags. Give three reasons to change anyway.

3. How many `<main>` elements can a page have? Why?
4. You have a box with three news items. Is the box a `<section>` or an `<article>`? And each news item?
5. What two things does a `<div class="btn">` fail to do that a `<button>` does?
6. When do you use `<button>` and when do you use `<a href>`?
7. Explain semantic HTML to a friend who is not technical, using the house and room names.

Try it

- **D2-01** Div soup → semantic: replace 7 `div class="…"` boxes. Use linked editing.
- **D2-02** 🐞 Semantic Misuse: 5 bugs on the TSI Blog. All of them are **R** or **V** (the Tab key).
- **D2-FF5** 🐞 Lost & Found: bugs 1–3 are semantic.
- **D2-HW1** 🐞 Haunted portfolio repair: bugs 2 and 3.

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. A good prompt: *"Here is my page plan: a list of three news items, each with a title and a date. Should the list be a section or an article, and each item? Explain why. Don't write the HTML for me."*

Go deeper

- MDN: HTML elements reference, "Content sectioning" (https://developer.mozilla.org/en-US/docs/Web/HTML/Element#content_sectioning)
- freeCodeCamp: Responsive Web Design → "Learn HTML by Building a Cat Photo App" (the semantic steps)

---

*Remember the One Rule: **You may only ship code you can explain, line by line.*** *Next note: 02 · Accessibility and meta: the parcel label read aloud.*

02 · Accessibility and Meta: The Parcel Label Read Aloud

File: notes/02-accessibility-and-meta.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2 · Day 2 · Block B2 **Layer:** Frontend (HTML for everyone)

Today in one sentence: write your page so that a person who cannot see it, cannot use a mouse, or reads it in another way still gets everything.

The picture to keep in your head

The parcel label read aloud. The `<head>` and its `<meta>` tags are **the label** on the parcel. A screen reader reads the label **and** the contents aloud to someone who cannot see them.

Imagine Didi ships a parcel of momo to a customer who is blind. A helper reads everything aloud to the customer:

- the label: "From Momo House. Language: English. Contents: steam momo, 10 pieces",
- then each thing inside: "A plate of ten steamed momo with red chutney".

If the label is blank, the helper says "Unknown parcel." If a photo has no description, the helper says "image… image… image". That is exactly what a screen reader does with a page that has no `lang`, no `<title>` and no `alt`.

Analogy (parcel read aloud)	Real thing (HTML)
"Language: English" on the label	`<html lang="en">`
"Contents: …" on the label	`<title>` and `<meta name="description">`
"Fits a small box / big box"	`<meta name="viewport">`
The little stamp on the corner	the favicon (`<link rel="icon">`)
Reading out the description of each item	`alt` on every `<img>`
"This is just packing paper, skip it"	`alt=""` on decorative images
Telling the person **where** a door leads, not "this door"	link text that makes sense alone
The printed word next to each box on a form	`<label>`

Words you now own

Term	What it means (plain English)	Example
Accessibility (a11y)	Making a page usable by **everyone**: blind users, keyboard-only users, people with low vision, slow internet, small phones. ("a11y" = a + 11 letters + y.)	A page you can use with only the Tab key
Screen reader	Software that reads the page aloud.	Windows **Narrator** (Ctrl+Win+Enter), NVDA, VoiceOver
`lang`	Tells the browser and screen reader which language the page is in, so it pronounces words correctly.	`<html lang="en">`, Nepali is `lang="ne"`
`alt` text	The description of an image, read aloud and shown if the image fails.	`alt="Didi serving momo at the counter"`
Decorative image	An image that adds no information (a border, a pattern).	`alt=""` (empty, but present)
Link text	The words inside `<a>`. Screen-reader users often list all links, so each must make sense **alone**.	"See the full menu", not "click here"
Label	The words linked to an input. Clicking the label focuses the input; screen readers read it.	`<label for="phone">Phone</label>`
Focus	The element the keyboard is on now. You see a **focus ring** around it.	Press Tab on any page
Contrast	How different the text colour is from the background. Low contrast is hard to read.	Grey `#aaa` on white = too low

Term	What it means (plain English)	Example
`<meta>`	Information **about** the page, in `<head>`. Not shown on the page.	description, viewport, charset
Favicon	The small icon on the browser tab.	`<link rel="icon" href="favicon.svg">`
Lighthouse	A DevTools report that scores your page (Accessibility, SEO…). Works offline.	F12 → Lighthouse → Analyze

How it works

Who is "everyone"?

- **Blind users** listen with a screen reader. They need `lang`, `alt`, good headings and good link text.
- **Keyboard users** (a broken mouse, a hand injury, power users) move with **Tab**, **Shift+Tab**, **Enter** and **Space**. They need real `<button>`, `<a>` and form elements.
- **Low-vision users** zoom in to 200 % and need strong contrast.
- **Everyone on a bad day**: a phone in the sun, a slow connection where images fail to load. `alt` text shows when an image is missing.

In Nepal and abroad, many government and bank sites must be accessible. Interviewers ask about `alt` and labels all the time.

alt text: three cases

The image is…	Write	Why
Information (a photo, a chart, a logo)	`alt="what it shows"`	The listener needs the same information
A link or button with no text (a logo that goes home)	`alt="Momo House home page"`	Say where it goes
Decoration only (a pattern, a divider)	`alt=""`	The screen reader skips it. Do **not** leave `alt` out: then it reads the file name

Good alt text is **short** (one sentence) and says what matters. Do not start with "image of" (the screen reader already says "image").

Link text must make sense alone

Screen-reader users can open a list of all links on a page. Imagine hearing: "click here, click here, click here". Useless. Compare: "Apply for the morning batch, see the fee table, download the brochure". Write the second kind.

Every input needs a label

A `placeholder` is **not** a label. It disappears when you type, it is often low contrast, and some screen readers skip it. Always use `<label for="…">` with a matching `id`.

Heading order is the table of contents

Screen-reader users jump from heading to heading, like a table of contents. So headings must go in order: `h1` → `h2` → `h3`. Never jump from `h1` to `h3` to make text smaller. (Size is CSS's job.)

The head: the parcel label

```
<head>
  <meta charset="UTF-8">                    → "the letters are UTF-8" (so Nepali text like नमस्ते works)
  <meta name="viewport" ...>                → "fit the phone screen; don't zoom out"
  <title>Momo House | Menu</title>          → tab text, bookmark name, Google result title, first thing read aloud
  <meta name="description" content="...">   → the grey text under the title in Google results
  <link rel="icon" href="favicon.svg">      → the tab icon
</head>
```

- **`<title>`**: describe *this* page. "Document" (the Emmet default) tells nobody anything.
- **`description`**: one or two sentences, about 150 characters. Google often shows it.
- **`viewport`**: without it, a phone shows the page zoomed out and tiny. Lighthouse 13 may not report it, so check with the device toolbar (F12 → Ctrl+Shift+M).

How to check: the Tab walk and Lighthouse

1. **The Tab walk.** Click on the page, then press **Tab** again and again. Every link, button and input should get a visible focus ring, in a sensible order. Anything you cannot reach is broken for keyboard users.
2. **Lighthouse.** F12 → **Lighthouse** tab → tick **Accessibility** (and **SEO** if you like) → **Analyze page load**. It lists problems such as "Image elements do not have [alt] attributes", "Links do not have a discernible name", "Form elements do not have associated labels", "`<html>` element does not have a [lang] attribute". It works offline.
3. **Narrator** (instructor demo only): Ctrl+Win+Enter turns it on and off. In class only the instructor uses it, with speakers, so 20 laptops don't talk at once.

Code you built today

Step 1: the admissions page before the audit

This is wrong on purpose. Read it and count the problems before you scroll on.

✗ Broken on purpose: find what is wrong

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Document</title>
</head>
<body>
  <h1>TSI Admissions</h1>
  <img src="images/tsi-building.svg">
  <img src="images/decorative-pattern.svg" alt="a picture of a pattern with lines and diamonds in red and blue">
  <p>Morning batch starts on Sunday. <a href="index.html">Click here</a> for the fee table.</p>
  <form>
    <input type="tel" id="phone" placeholder="Phone number">
  </form>
</body>
</html>
```

Predict: run Lighthouse on this page in your head. Which four problems will it list? Which two will it *not* list, because only a human can judge them?

Step 2: the same page after the audit

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Join the AI-Powered Full Stack Developer Program at Tech Skills Institute,
Dillibazar, Kathmandu. Morning and evening batches.">
  <title>TSI Admissions | AI-Powered Full Stack Developer Program</title>
  <link rel="icon" href="images/favicon.svg" type="image/svg+xml">
</head>
<body>
  <header>
    <h1>TSI Admissions</h1>
  </header>
  <main>
    <img src="images/tsi-building.svg" alt="The TSI building in Dillibazar, Kathmandu" width="400">
    <img src="images/decorative-pattern.svg" alt="" width="400">
    <h2>Batches</h2>
    <p>The morning batch starts on Sunday. <a href="index.html">See the fee table</a>.</p>
    <h2>Call me back</h2>
    <form method="get">
      <label for="phone">Phone number</label>
      <input type="tel" id="phone" name="phone" placeholder="98XXXXXXXX">
      <button type="submit">Send</button>
    </form>
  </main>
</body>
</html>
```

- `lang="en"`: the screen reader uses English pronunciation.
- A real `<title>` and `<meta name="description">`: the label says what is inside.
- The building photo says what it shows. The pattern has `alt=""`, so it is skipped.
- "See the fee table" makes sense alone. "Click here" does not.
- The phone input has a `<label>`. The placeholder is now only an example of the format.

Predict: run Lighthouse → Accessibility on Step 2. Is the score higher? Which items moved to "Passed audits"?

Step 3: a logo that is also a link

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Momo House in Thamel: steam, fried and jhol momo made fresh every day by Didi.">
  <title>Momo House | Home</title>
  <link rel="icon" href="images/favicon.svg" type="image/svg+xml">
</head>
<body>
  <header>
    <a href="index.html"><img src="images/logo-momo-house.svg" alt="Momo House home page" width="80"></a>
    <nav>
      <a href="index.html">Home</a> |
      <a href="index.html">Our menu</a> |
      <a href="index.html">Order online</a>
    </nav>
  </header>
  <main>
    <h1>Fresh momo in Thamel</h1>
    <h2>Opening hours</h2>
    <p>Sunday to Friday, 10:00 to 21:00.</p>
    <h3>Saturday</h3>
    <p>Closed. Didi visits her family in Patan.</p>
  </main>
</body>
</html>
```

- The logo is the only thing inside the link, so its `alt` says **where the link goes**.
- Headings go h1 → h2 → h3, never skipping a level.

Predict: press Tab from the top. What is the first thing that gets focus?

Quick reference

Check	Good	Bad	
Language	`<html lang="en">`	`<html>`	
Title	`Momo House	Home`	Menu`	`Document`
Informative image	`alt="Didi at the momo counter"`	no `alt`, or `alt="image"`	
Decorative image	`alt=""`	a long description of the decoration	
Link text	"See the full menu"	"click here", "read more", "link"	
Input	`<label for="phone">` + `id="phone"`	a placeholder only	
Headings	h1 → h2 → h3	h1 → h3 (to make it smaller)	
Phone screens	`<meta name="viewport" content="width=device-width, initial-scale=1.0">`	none	
Search result text	`<meta name="description" content="…">`	none	
Contrast	dark navy `#14213D` on white	light grey `#bbbbbb` on white	

Mistakes everyone makes

You see…	Because…	Fix	Planted in
Lighthouse: "`<html>` element does not have a [lang] attribute"	`lang` missing	`<html lang="en">`	D2-03 bug 1

You see…	Because…	Fix	Planted in
The tab says "Document"	Emmet's default title was never changed	A title that names the page and site	D2-03 bug 2
Lighthouse: "Image elements do not have [alt] attributes"; HTMLHint underlines the img	No `alt`	Describe the image	D2-03 bug 3, D2-HW1 bug 4
The screen reader reads a long story about a border	A decorative image has a long alt	`alt=""`	D2-03 bug 4
Three links all say "click here". Lighthouse may stay silent (it skips links to `#…` on the same page), so this is an **R** bug: read the link text	Vague link text	Say where the link goes	D2-03 bug 5 (R)
Clicking the word next to the box does nothing, and the hint vanishes when you type. Lighthouse accepts a placeholder, so don't trust one tool	Placeholder only, no `<label>`	Add `<label for>` + `id`	D2-03 bug 6 (V)
Lighthouse: "Heading elements are not in a sequentially-descending order"	h1 → h3 → h2	Keep the order; use CSS for size	D2-FF5 bug 4
The Tab key skips the "Apply" button	`<div class="btn">`	`<button>`	D2-FF5 bug 5
A fee table with bold first row but no headers	`<b>` instead of `<th>`	Use `<th scope="col">`	D2-FF5 bug 6
Lighthouse (SEO): "Document does not have a meta description"	No description	Add `<meta name="description">`	D2-HW4 bug 1
On a phone (F12 → device toolbar, Ctrl+Shift+M) the page is tiny and you must zoom. Lighthouse 13 calls the check "not applicable", so it is an **R** bug	No viewport	Add the viewport meta	D2-HW4 bug 2 (R)

Check yourself

1. What does "a11y" stand for, and why is it written that way?
2. Name three groups of people who need an accessible page.
3. When do you write `alt=""`, and why is that different from no `alt` at all?
4. Why is "click here" bad link text? Write better text for a link to the fee table.
5. Give two reasons a placeholder is not enough instead of a label.
6. What does the viewport meta tag do on a phone?
7. Describe the Tab walk test in three steps.
8. Explain to a friend why a blind person needs `lang` and `alt`, using the parcel read aloud.

Try it

- **D2-03** A11y Audit: TSI Admissions (6 bugs). Check with the Tab walk and Lighthouse.
- **D2-FF5** Lost & Found: bugs 4–7.
- **D2-HW4** Patan Craft Shop replay: bugs 1–2.
- **D2-HW2**: your portfolio must reach Lighthouse Accessibility ≥ 90.

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. A good prompt: *"Lighthouse says 'Form elements do not have associated labels' on my page. Here is my form. Explain what it means and how a screen reader uses a label. Don't rewrite my code."*

Go deeper

- MDN: HTML: A good basis for accessibility (https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Accessibility/HTML)
- web.dev: Learn Accessibility (https://web.dev/learn/accessibility)

*Remember the One Rule: **You may only ship code you can explain, line by line.*** *Next note: 03 · Audio, video, iframe: TV, radio, and a window into another room.*

03 · Audio, Video and iframe: TV, Radio, and a Window into Another Room

File: notes/03-audio-video-iframe.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2 · Day 2 · Block B3 **Layer:** Frontend (HTML media)

Today in one sentence: `<audio>` is a radio, `<video>` is a TV, and `<iframe>` is a window that shows another page inside yours.

The picture to keep in your head

TV, radio, and a window into another room.

Analogy	Real thing	What you must give it
A **radio** on the canteen shelf	`<audio>`	a sound file, and **buttons** (`controls`) so people can switch it on
A **TV** on the wall	`<video>`	a video file, buttons (`controls`), a picture to show before it plays (`poster`)
The **channel** the TV is tuned to	`<source src="…" type="…">`	the right file and the right type
"If this TV is broken, read the menu on the board"	fallback text inside `<video>` / `<audio>`	a sentence for old browsers
A **window** into the next room	`<iframe>`	the page to show (`src`), a size, and a **title** (what room is it?)

A radio with no buttons is still a radio, but nobody can switch it on. That is `<audio>` without `controls`: it exists, but you see **nothing**.

Words you now own

Term	What it means (plain English)	Example
`<audio>`	Plays sound.	`<audio src="media/canteen-bell.wav" controls></audio>`
`<video>`	Plays video.	`<video src="media/momo-steam.mp4" controls width="640"></video>`
`controls`	A **boolean attribute**: just writing it turns on the play/pause/volume bar.	`controls` (no value needed)
Boolean attribute	An attribute that is ON when it is present and OFF when it is missing. `controls="false"` is still ON!	`controls`, `muted`, `autoplay`, `loop`, `required`
`<source>`	One file option for a video or audio. Goes **inside** `<video>`/`<audio>`.	`<source src="media/momo-steam.mp4" type="video/mp4">`
`type` (MIME type)	Tells the browser what kind of file it is, so it can skip ones it cannot play.	`video/mp4`, `audio/wav`, `audio/mpeg`
`poster`	The picture a video shows before it plays.	`poster="images/poster-momo-steam.svg"`
`autoplay`	Start playing by itself. Browsers allow it only when the video is `muted` (or the file has no sound track).	`autoplay muted`
`muted`	Start with the sound off.	`muted`
`loop`	Start again at the end.	`loop`
Fallback text	Text inside `<video>`/`<audio>` that only very old browsers show.	"Your browser cannot play this video."
`<iframe>`	Shows another web page inside a box on your page.	`<iframe src="kathmandu-map.html" title="Map of Kathmandu"></iframe>`
`title` (on iframe)	Tells a screen reader what is in the window. Required for accessibility.	`title="Map of Momo House in Thamel"`

How it works

Audio and video are boxes, not stickers

`<img>` is a sticker (no closing tag). `<audio>` and `<video>` are **boxes**: they have a closing tag, because they can hold `<source>` tags and fallback text inside.

```
<video controls width="640" poster="images/poster-momo-steam.svg">
  <source src="media/momo-steam.mp4" type="video/mp4">    ← channel option 1
  Your browser cannot play this video.                    ← fallback text
</video>
```

src on the tag, or source inside?

Both work:

- **Short form:** `<video src="media/momo-steam.mp4" controls></video>` for one file.
- **Long form:** one or more `<source>` tags **inside**. The browser tries them from top to bottom and plays the first one it can. The `type` lets it skip files it knows it cannot play, without downloading them.

If you put `<source>` **after** `</video>`, it is not inside the TV any more. The TV has no channel, so the video stays empty.

Common file types

File	Use	type
`.mp4` (H.264)	video, works in every modern browser	`video/mp4`
`.webm`	video, smaller files	`video/webm`
`.mp3`	audio	`audio/mpeg`
`.wav`	audio, big but simple	`audio/wav`

A wrong `type` (for example `video/mp3` on an MP4 file) makes the browser think "I can't play that", and it **skips** the file.

The autoplay rule

Nobody likes a page that shouts at them. So browsers block **autoplay with sound**. They allow `autoplay` when the video is also `muted`:

```
<video src="media/momo-steam.mp4" autoplay muted loop width="320"></video>
```

Use autoplay rarely (a short silent background loop at most). Always give `controls` for anything with sound. Our class checker (html-validate, with its accessibility rules) even warns about **every** `autoplay`, muted or not: "The autoplay attribute is not allowed on `<video>`". Moving things distract some readers and make others feel unwell. So the safe habit is: **no autoplay; give `controls` and let the user choose.**

Boolean attributes: present = ON

`controls="false"` does **not** hide the controls. The browser only checks "is the word `controls` there?". To turn it off, delete the attribute.

iframe: a window into another room

`<iframe>` shows a whole other page inside a box. It is used for maps, YouTube videos, payment forms and more. In class we show a **local** page (`kathmandu-map.html`), so it works without Wi-Fi.

- `src`: which page to show. A wrong name shows a 404 page **inside** the window.
- `width` and `height`: the size of the window, in pixels (no `px` in HTML attributes).
- `title`: says what the window shows. Screen readers read it. Lighthouse reports a missing title.

Many big sites (Google, banks) refuse to be shown in someone else's iframe, for safety. That is why a YouTube link must use its special "embed" address. For now, use your own local pages.

Paths are the same as on Day 1

`media/canteen-bell.wav` means "the `media` folder next to this page". Spelling and case must match the real file name exactly: `canteen_bell.wav` (underscore) is a different file from `canteen-bell.wav`.

Code you built today

Step 1: the canteen bell (audio)

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TSI Canteen | The lunch bell</title>
</head>
<body>
  <main>
    <h1>The TSI Canteen bell</h1>
    <p>Didi rings this bell when lunch is ready.</p>
    <audio src="media/canteen-bell.wav" controls>
      Your browser cannot play audio. The bell means: lunch is ready!
    </audio>
  </main>
</body>
</html>
```

- `controls` shows the play bar. Without it, the `<audio>` is on the page but invisible.

Predict: delete `controls` and save. What do you see? Is the audio element still in the Elements panel?

Step 2: the momo steam video

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House | How we steam momo</title>
</head>
<body>
  <main>
    <h1>How Didi steams momo</h1>
    <video controls width="640" poster="images/poster-momo-steam.svg">
      <source src="media/momo-steam.mp4" type="video/mp4">
      Your browser cannot play this video. Visit Momo House in Thamel to see it live!
    </video>
    <p>Four seconds of steam. Ten minutes in real life.</p>
    <h2>Watch it again and again</h2>
    <video src="media/momo-steam.mp4" controls loop width="320"></video>
  </main>
</body>
</html>
```

- The first video starts with the **poster** picture and waits for the user.
- The second one has no poster, so it shows its first frame. It starts again at the end because of `loop`.
- Try it yourself: add `autoplay muted` to the second video. It plays by itself. On a video **with sound**, removing `muted` usually stops autoplay until the visitor has clicked on the site. (Our momo video has no sound track, so Chrome plays it anyway: that is why the class bug-hunts never test autoplay.)

Predict: change `poster="images/poster-momo-steam.svg"` to `poster="image/poster-momo-steam.svg"` (no "s"). What does the first video look like before you press play?

Step 3: a window into the map page

File: `kathmandu-map.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Kathmandu map</title>
</head>
<body>
  <h1>Find us in Kathmandu</h1>
  <img src="images/kathmandu-map.svg" alt="Map of Kathmandu with the Ring Road and pins for TSI Dillibazar, Thamel,
Swayambhu, Patan and Boudha" width="500">
</body>
</html>
```

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House | Find us</title>
</head>
<body>
  <main>
    <h1>Find Momo House</h1>
    <p>We are in Thamel, a short walk from TSI.</p>
    <iframe src="kathmandu-map.html" title="Map of Kathmandu showing Momo House and TSI" width="600"
height="450"></iframe>
  </main>
</body>
</html>
```

- `index.html` shows `kathmandu-map.html` inside a 600 × 450 window.
- `title` tells a screen reader what the window shows.

Predict: change `src` to `kathmandu-map.htm` (no "l"). Where does the error appear: on your page or inside the window?

Quick reference

```
<audio src="media/canteen-bell.wav" controls></audio>

<video controls width="640" poster="images/poster-momo-steam.svg">
  <source src="media/momo-steam.mp4" type="video/mp4">
  Your browser cannot play this video.
</video>

<video src="media/momo-steam.mp4" autoplay muted loop width="320"></video>   (only if you really must)

<iframe src="kathmandu-map.html" title="Map of Kathmandu" width="600" height="450"></iframe>
```

Attribute	On	Does
`controls`	audio, video	shows play/pause/volume
`poster`	video	picture before play
`autoplay` + `muted`	video	plays by itself, silently
`loop`	audio, video	repeats
`width` / `height`	video, iframe	size in pixels (no `px`)
`title`	iframe	names the window for screen readers

Mistakes everyone makes

You see…	Because…	Fix	Planted in
Nothing at all where the audio should be	No `controls`	Add `controls`	D2-04 bug 1

You see…	Because…	Fix	Planted in
The player shows 0:00 and will not play	File name wrong (`canteen_bell.wav`)	Match the real name: `canteen-bell.wav`	D2-04 bug 2
A still frame with no play button	`<video>` has no `controls`	Add `controls`	D2-04 bug 3
A black box before play	`poster` path wrong (`image/` instead of `images/`)	Fix the folder name	D2-04 bug 4
"Not found" **inside** the iframe	Wrong file name or extension (`.htm`)	`kathmandu-map.html`	D2-04 bug 5
An empty video player	`<source>` is after `</video>`	Move `<source>` inside	D2-HW4 bug 3
The second video never plays	`type="video/mp3"` on an MP4 file	`type="video/mp4"`	D2-HW4 bug 4
Lighthouse: "`<frame>` or `<iframe>` elements do not have a title"	No `title`	Add a `title`	D2-FF5 bug 7
Controls still show with `controls="false"`	Boolean attributes are ON when present	Remove the attribute	(class talk)

Check yourself

1. Why do `<audio>` and `<video>` need a closing tag, when `<img>` does not?
2. What does the `controls` attribute do? What happens without it?
3. Why does `controls="false"` still show the controls?
4. What is a `poster`, and when does the user see it?
5. Under which condition will a browser let a video autoplay?
6. Why should every `<iframe>` have a `title`?
7. The video player is empty but the file exists. Name two possible reasons.
8. Explain `<iframe>` to a friend using "a window into another room".

Try it

- **D2-04** 🐞 Broken Media (5 bugs, all V).
- **D2-05** Media corner: the audio and video are done; add the iframe with a `title`, `width` and `height`. Bonus: fallback text.
- **D2-HW4** 🐞 Patan Craft Shop replay: bugs 3 and 4.
- **D2-FF5** 🐞 Lost & Found: bug 7.

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. A good prompt: *"My video shows only a black box with a play button, and nothing happens when I press it. Here is my video tag and my folder list. What are the possible causes? Explain; don't rewrite it."*

Go deeper

- MDN: Video and audio content (https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content/HTML_video_and_audio)
- MDN: `<iframe>` (https://developer.mozilla.org/en-US/docs/Web/HTML/Element/iframe)

---

*Remember the One Rule: **You may only ship code you can explain, line by line.*** *Next note: 04 · CSS basics and linking: the skeleton gets its outfit.*

04 · CSS Basics: The Outfit, Rule Anatomy and Three Ways to Add CSS

File: notes/04-css-basics-and-linking.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2 · Day 2 · Blocks B0 + B4 **Layer:** Frontend (CSS = style)

Today in one sentence: HTML is the skeleton; CSS is the outfit, and one stylesheet file can dress every page of your site.

The picture to keep in your head

Skeleton & outfit (the Zen Garden idea). In B0 you saw **one** HTML page wearing **three** different stylesheets: traditional, modern and night. The HTML did not change at all. Only one `href` changed. Same skeleton, new outfit.

Then, for the three ways to add CSS: **Outfit**: sticky note / outfit list / **school uniform**.

Analogy	Real thing	Reach
A **sticky note** on one shirt: "make this shirt red"	**inline** CSS: `style="…"` on one tag	one element
An **outfit list** for one person: "today Aarav wears…"	**internal** CSS: a `<style>` block in the `<head>`	one page
The **school uniform**: one rule for every student	**external** CSS: a `.css` file linked with `<link>`	every page that links it

At TSI, imagine changing the uniform colour. With a school uniform rule, you change **one** list and every student changes. With sticky notes, you would need a note on every single shirt. That is why real sites use **external** CSS.

Words you now own

Term	What it means (plain English)	Example
CSS	Cascading Style Sheets: the language for how a page **looks**.	colours, fonts, sizes, spacing
Rule	One instruction: "these elements → look like this".	`h1 { color: maroon; }`
Selector	**Which** elements the rule is for.	`h1`, `.price`, `#menu`
Declaration block	Everything between `{` and `}`.	`{ color: maroon; font-size: 2rem; }`
Declaration	One `property: value;` pair.	`color: maroon;`
Property	**What** you change.	`color`, `background-color`, `font-size`
Value	**To what** you change it.	`maroon`, `#14213D`, `2rem`
Inline style	CSS in a `style` attribute on one tag.	`<p style="color: red;">`
Internal stylesheet	CSS inside `<style>` … `</style>` in the `<head>`.	for a one-page demo
External stylesheet	CSS in its own file, linked from the `<head>`.	`<link rel="stylesheet" href="css/style.css">`
Comment	A note the browser ignores. In CSS **only** `/* … */` works.	`/* footer styles */`
Cascade	When two rules disagree, the browser picks a winner. With equal selectors, **the last rule wins**.	two `h1` rules → the later one wins
Styles panel	The DevTools pane that shows every rule on the selected element, and which ones lost (crossed out).	F12 → Elements → Styles

How it works

Anatomy of a rule

```
selector      declaration block
   │       ┌──────────────┴──────────────┐
   h1      {  color: maroon;  font-size: 2rem;  }
              └──┬──┘  └──┬──┘
            property   value
              └─── declaration ──┘  ← ends with ;
```

Four tiny pieces of punctuation hold it together:

- `{` opens the block and `}` closes it. **Every** `{` needs its `}`.
- `:` sits between the property and the value.
- `;` ends **each** declaration. (You may leave out the last `;`. Write it anyway. Then you can add a line later without breaking anything.)

The three ways, side by side

```
INLINE (sticky note)            INTERNAL (outfit list)          EXTERNAL (school uniform)
<p style="color: red;">         <head>                          <head>
                                  <style>                         <link rel="stylesheet"
                                    p { color: red; }                   href="css/style.css">
                                  </style>                        </head>
                                </head>                           ── css/style.css ──
                                                                  p { color: red; }
one element                     one page                        every page that links it
```

Use **external** for real work. Use internal for a quick one-page test. Use inline almost never (it is hard to find and hard to override).

The link tag, letter by letter

```
<link rel="stylesheet" href="css/style.css">
      │                │
      │                └─ the path to the file, from THIS html file (same path rules as Day 1)
      └─ "this file is a stylesheet". Spell it exactly: stylesheet (one word, all small letters)
```

- `<link>` goes in the `<head>`. It is a void tag (no closing tag), like `<img>`.
- If `rel` is misspelt (`stylsheet`), the browser does not know the file is CSS. **Nothing** loads.
- If `href` is wrong (`styles.css` instead of `style.css`), the browser asks for a file that does not exist. **Nothing** loads. F12 → Network shows the file in red with **404**.

The cascade: last rule wins (for now)

When two rules with the **same kind of selector** set the same property, the rule that comes **later** wins:

```
h1 { color: navy; }
h1 { color: maroon; }
```

The `h1` is maroon. In note 05 you learn the second part of the cascade: a more **specific** selector (like `#id`) beats a less specific one, wherever it is.

Order across files counts too: if a page links `a.css` and then `b.css`, the rules in `b.css` come later.

DevTools Styles: your X-ray

1. Right-click any element → **Inspect**.
2. The **Styles** pane on the right lists every rule that matches, newest/strongest first.
3. A declaration that **lost** is ~~crossed out~~. A declaration the browser **did not understand** is crossed out with a yellow ⚠ warning triangle ("Invalid property value" or "Unknown property name").
4. You can click any value and type a new one to test. It is temporary: copy the good value back into your file.

Comments

```
/* Momo House: header styles */
header { background-color: #14213D; }
```

Only `/* … */` is a comment in CSS. `// like this` is JavaScript style. In CSS it is not a comment: it breaks the rule that follows it (see note 09).

Code you built today

Step 1: the outfit swap (B0, D2-00)

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House | Outfit swap</title>
  <link rel="stylesheet" href="css/outfit-traditional.css">
</head>
<body>
  <h1>Momo House</h1>
  <p>Fresh momo, made by Didi in Thamel since 2010.</p>
  <p><a href="index.html">See the menu</a></p>
</body>
</html>
```

File: `css/outfit-traditional.css`

```
body {
  background-color: #fdf6ec;
  color: #5a1a00;
  font-family: Georgia, "Times New Roman", serif;
}

h1 {
  color: #990011;
}
```

File: `css/outfit-night.css`

```
body {
  background-color: #14213D;
  color: #f5f5f5;
  font-family: Arial, Helvetica, sans-serif;
}

h1 {
  color: #fca311;
}

a {
  color: #fca311;
}
```

- Change `outfit-traditional.css` to `outfit-night.css` in the `<link>`, save, and the whole page changes. The HTML did not change.

Predict: you link **both** files, traditional first, night second. What colour is the `h1`? Why?

Step 2: a sticky note (inline)

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sunita's bio | Inline</title>
</head>
<body>
  <h1 style="color: maroon;">Sunita Gurung</h1>
  <p>Sunita is a student at TSI in Dillibazar.</p>
  <p>She likes jhol momo and long walks to Swayambhu.</p>
</body>
</html>
```

- Only this one `h1` changes. A second `h1` on the page would not.

Predict: you want every paragraph to be grey. How many sticky notes do you need on this page? On a site with 40 pages?

Step 3: an outfit list (internal)

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sunita's bio | Internal</title>
  <style>
    h1 {
      color: maroon;
    }

    p {
      color: #555555;
    }
  </style>
</head>
<body>
  <h1>Sunita Gurung</h1>
  <p>Sunita is a student at TSI in Dillibazar.</p>
  <p>She likes jhol momo and long walks to Swayambhu.</p>
</body>
</html>
```

- One rule now styles **every** paragraph on this page. But another page would not get it.

Step 4: the school uniform (external)

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sunita's bio</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Sunita Gurung</h1>
  <p>Sunita is a student at TSI in Dillibazar.</p>
  <p>She likes jhol momo and long walks to Swayambhu.</p>
  <p><a href="about.html">More about Sunita</a></p>
</body>
</html>
```

File: `about.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>About Sunita</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>About Sunita</h1>
  <p>Sunita wants to build websites for trekking companies in Pokhara.</p>
  <p><a href="index.html">Back to the bio</a></p>
</body>
</html>
```

File: `css/style.css`

```
/* Sunita's site: one uniform for every page */
body {
  background-color: #fdf6ec;
  font-family: Arial, Helvetica, sans-serif;
}

h1 {
  color: maroon;
}

p {
  color: #555555;
  font-size: 18px;
}

a {
  color: #14213D;
}
```

- Both pages link the **same** file. Change `maroon` to `navy` once, and both pages change.
- The path `css/style.css` means "the `css` folder next to this page".

Predict: you add `h1 { color: green; }` at the **end** of `style.css`. What colour are the headings now?

Quick reference

You want to…	Write
Link a stylesheet	`<link rel="stylesheet" href="css/style.css">` (in `<head>`)
Write a rule	`selector { property: value; }`
Add a comment	`/* like this */`
Style one element only, quickly	`style="color: red;"` (avoid)
Style one page for a test	`<style> … </style>` in `<head>`
See which rules apply	Right-click → Inspect → Styles
See if the CSS file loaded	F12 → Network → refresh → look for `style.css` (200 = OK, 404 = not found)

Mistakes everyone makes

You see…	Because…	Fix	Planted in
No CSS at all	`rel="stylsheet"` typo	`rel="stylesheet"`	D2-06 bug 1
No CSS at all; Network shows **404** for the CSS file	`href="css/styles.css"`, but the file is `style.css`	Match the real file name	D2-06 bug 2
No CSS; the HTML is in the root, the CSS in `css/`	`href="style.css"` misses the folder	`href="css/style.css"`	D2-HW1 bug 1
One rule works, the next one doesn't	A missing `}`	Close every `{`	D2-06 bug 3, D2-HW1 bug 10, D2-FF4

You see…	Because…	Fix	Planted in
Both declarations are dropped: the browser reads them as one broken value	A missing `;` between them	End every declaration with `;`	D2-06 bug 4, D2-FF4
The text stays black; DevTools says "Unknown property name"	`colour: maroon;` (British spelling)	CSS uses US spelling: `color`	D2-06 bug 5, D2-FF4
The rule after a comment is ignored	`// footer styles`	`/* footer styles */`	D2-06 bug 6, D2-FF4
A rule is crossed out in DevTools, no warning	A later rule (or a stronger selector) won	Remove one, or change the order	(the cascade)

Fix the link first: until the stylesheet loads, no other CSS mistake is visible. The D2-06 README says the same.

Check yourself

1. Label the parts of `p { color: grey; }`: selector, property, value, declaration, declaration block.
2. Name the three ways to add CSS. Which one would a real 10-page site use, and why?
3. Two rules set `h1` colour: `navy` first, `maroon` second. Which wins? What is this called?
4. Your page has no style at all. Name three things you check, in order.
5. What does DevTools show for a declaration the browser did not understand?
6. Why is `// comment` dangerous in a CSS file?
7. Explain the school uniform analogy to a friend in two sentences.

Try it

- **D2-00** Outfit swap: change one `href`, get a new look.
- **D2-06** Stylesheet Not Loading (6 bugs). Fix bugs 1 and 2 first.
- **D2-07** Restyle the bio: change 4 things in Sunita's CSS.
- **D2-15** Style Momo House, checkpoint 1: link `css/style.css` from all 4 pages.

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. A good prompt: *"My page shows no CSS at all. Here is my link tag and a list of my folders and files. Explain the three most common reasons a stylesheet does not load and how to check each one in DevTools. Don't fix it for me."*

Go deeper

- MDN: CSS first steps: what is CSS? (https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/What_is_CSS)
- The CSS Zen Garden (Wi-Fi): csszengarden.com: one HTML page, hundreds of outfits.

---

*Remember the One Rule: **You may only ship code you can explain, line by line.*** *Next note: 05 · Selectors: calling students in class.*

05 · Selectors: Calling Students in Class

File: notes/05-selectors.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2 · Day 2 · Block B5 **Layer:** Frontend (CSS)

Today in one sentence: a selector is how you *call* the elements you want to style: everyone, a team, one person by name, or a team in one row.

The picture to keep in your head

Calling students in class. The instructor stands at the front of the TSI classroom:

The instructor says…	Who stands up	CSS selector	Example
"**Everyone**, stand up."	every student	**element** selector	`p { … }` = every paragraph
"**Red team**, stand up."	everyone wearing a red-team badge	**class** selector	`.veg { … }` = every element with `class="veg"`
"**Aarav, by name**, stand up."	exactly one person	**id** selector	`#todays-special { … }` = the one element with that id
"**Red team in row 2**, stand up."	red-team students who sit **inside** row 2	**descendant** selector	`nav a { … }` = links inside a nav
"Red team **and** blue team, stand up."	both groups, same instruction	**grouping**	`h1, h2 { … }`
"Students on the red team **who are also** captains."	only people with **both** badges	**two classes together**	`.dish.spicy { … }`
"Whoever I am pointing at right now."	changes as the finger moves	`:hover`	`a:hover { … }`

Aarav is on the red team. The instructor says "Aarav, stand up" and also "Red team, stay seated". Aarav stands up: he listens to his **name**, because a name is more specific than a team. CSS works the same way (specificity, below).

Words you now own

Term	What it means (plain English)	Example
Element (type) selector	Matches every tag of that name.	`p`, `h1`, `a`, `li`
Class	A label you can put on **many** elements. An element can have many classes, separated by **spaces**.	`class="dish veg"`
Class selector	A **dot** + the class name.	`.veg`
id	A name for **one** element. Unique on the page.	`id="todays-special"`
id selector	A **hash** `#` + the id.	`#todays-special`
Grouping	One rule for several selectors, separated by **commas**.	`h1, h2, h3 { … }`
Descendant selector	Two selectors with a **space**: the second **inside** the first (at any depth).	`nav a`
Compound (`.a.b`)	Two classes with **no space**: one element that has **both**.	`.dish.spicy`
Pseudo-class	A colon + a state word.	`:hover` (the mouse is over it)
Specificity	The score that decides which rule wins when two rules disagree.	id beats class beats element
Case-sensitive	`Card` and `card` are different names.	`.card` does not match `class="Card"`

How it works

The punctuation is the whole language

```
p            no symbol   → element  ("everyone who is a p")
.price       dot         → class    ("everyone with the price badge")
#menu        hash        → id       ("the one called menu")
h1, h2       comma       → grouping ("h1s AND h2s")
nav a        space       → descendant ("a's that are INSIDE a nav")
.dish.spicy  no space    → compound ("one element with BOTH classes")
a:hover      colon       → state    ("an a while the mouse is on it")
```

One missing dot or one extra space changes the meaning completely. Read selectors slowly, symbol by symbol.

Class vs id

	class	id
How many elements can have it?	many	**one** per page
How many can one element have?	many (`class="dish veg spicy"`)	one
In CSS	`.name`	`#name`
Used for	styling groups	one special element, and `#anchor` links (Day 1)
Strength	medium	strong

Habit: style with **classes**. Keep ids for the few unique things (and for links like `href="#menu"`).

Space or no space? `.btn .primary` vs `.btn.primary`

```
.btn .primary    (space)    → an element with class "primary" INSIDE an element with class "btn"
                              <div class="btn"><span class="primary">…</span></div>   ✔ matches the span
.btn.primary     (no space) → ONE element with BOTH classes
                              <button class="btn primary">…</button>                  ✔ matches the button
```

Classes in HTML: spaces, not commas

`class="card special"` gives the element **two** classes: `card` and `special`. `class="card, special"` gives it the classes `card,` (with a comma!) and `special`. Now `.card` does **not** match. HTML separates classes with spaces only.

Names must match exactly

The selector and the HTML must match letter for letter, **including capital letters**: `.card` ≠ `class="Card"`, `#menu` ≠ `id="main-menu"`. When a rule "does nothing", first compare the two names side by side.

Light specificity: who wins?

When two rules set the same property on the same element:

1. **The stronger selector wins**: `#id` (strong) > `.class` (medium) > `element` (weak).
2. If they are **equally** strong, the **later** rule wins (note 04).

```
p              { color: blue; }    weak
.note          { color: green; }   medium
#only          { color: red; }     strong     → <p class="note" id="only"> is RED
                                                 → <p class="note">           is GREEN
                                                 → <p>                        is BLUE
```

(We checked this in Chrome. The order of the three rules does not matter here, because their strengths are different.)

You will learn the exact scoring (and why you avoid `!important`) in session 3.2. For now: **id > class > element, then last wins.**

`:hover`

`a:hover { … }` applies only while the mouse is over the link. Write the colon **directly** after the selector, with no space: `a:hover`, not `a :hover`. A typo like `a:hoverr` is an unknown state, and the **whole rule is dropped** (note 09).

Code you built today

Step 1: element, class and id

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House | Menu</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Momo House menu</h1>
  <p>All momo come with tomato and sesame chutney.</p>
  <p class="dish veg">Veg momo, Rs. 120</p>
  <p class="dish">Chicken momo, Rs. 150</p>
  <p class="dish veg">Paneer momo, Rs. 160</p>
  <p class="dish" id="todays-special">Today's special: jhol momo, Rs. 200</p>
</body>
</html>
```

File: `css/style.css`

```
/* everyone */
p {
  color: #555555;
}

/* the veg team */
.veg {
  color: green;
}

/* one dish, by name */
#todays-special {
  font-weight: bold;
  font-size: 22px;
}
```

- `p` makes every paragraph grey. `.veg` makes the two veg dishes green. `#todays-special` makes one dish bold and bigger.
- The veg dishes are green, not grey: `.veg` (a class) is stronger than `p` (an element).

Predict: add `class="veg"` to the jhol momo line. What colour is it? Is it still bold?

Step 2: grouping, descendant and two classes

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House | Menu</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <nav>
    <a href="index.html">Home</a>
    <a href="index.html">Menu</a>
    <a href="index.html">Order</a>
  </nav>
  <h1>Momo House menu</h1>
  <h2>Steam</h2>
  <p class="dish">Chicken momo, Rs. 150</p>
  <p class="dish spicy">Chilli momo, Rs. 180</p>
  <h2>Contact</h2>
  <p>Questions? <a href="index.html">Talk to Didi</a>.</p>
</body>
</html>
```

File: `css/style.css`

```
/* two groups, one instruction */
h1,
h2 {
  color: #990011;
  font-family: Arial, Helvetica, sans-serif;
}

/* only links INSIDE the nav */
nav a {
  text-decoration: none;
  margin-right: 12px;
}

/* only elements with BOTH classes */
.dish.spicy {
  border: 2px solid red;
}
```

- `h1, h2` styles both heading levels with one rule.
- `nav a` removes the underline from the three menu links, but **not** from "Talk to Didi", which is outside the nav.
- `.dish.spicy` puts a red border on the chilli momo only.

Predict: you change `nav a` to `nav, a`. Which elements lose their underline now, and what else changes?

Step 3: hover and the strength test

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Selector strength</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Who wins?</h1>
  <p>Plain paragraph.</p>
  <p class="note">Paragraph with a class.</p>
  <p class="note" id="only">Paragraph with a class and an id.</p>
  <p><a href="index.html">Hover over me</a></p>
</body>
</html>
```

File: `css/style.css`

```
#only {
  color: red;
}

.note {
  color: green;
}

p {
  color: blue;
}

a:hover {
  color: white;
  background-color: #14213D;
}
```

- The rules are in "wrong" order on purpose: `#only` comes first, but it still wins on its paragraph. Strength beats order.
- Hover over the link: it turns white on navy.

Predict: before you open it, write the colour of each paragraph. Then check with F12 → Styles: which declarations are crossed out?

Quick reference

Selector	Matches	Example
`p`	every `<p>`	`p { color: grey; }`
`.veg`	every element with class `veg`	`<li class="veg">`
`#cart`	the one element with `id="cart"`	`<div id="cart">`
`h1, h2`	every h1 **and** every h2	grouping
`nav a`	every `a` inside a `nav`	descendant
`.dish.spicy`	elements with **both** classes	`<p class="dish spicy">`
`a:hover`	an `a` while the mouse is on it	state
Strength	`#id` > `.class` > `element`; equal → last wins	

Mistakes everyone makes

You see…	Because…	Fix	Planted in
The rule does nothing	`.card` in CSS, `class="Card"` in HTML	Same case in both	D2-08 bug 1, D2-HW1 bug 6, D2-FF4 (`.Title`)
The rule does nothing	`#menu` in CSS, `id="main-menu"` in HTML	Same name in both	D2-08 bug 2, D2-FF4 (`#nav`), D2-HW1 bug 12 (`#footer` vs `class="footer"`), D2-HW4 bug 5 (`#Header`)
Prices are not styled	`price { … }` missing its dot	`.price { … }`	D2-08 bug 3
Neither h1 nor h2 is styled	`h1, h2, { … }`: a trailing comma makes the selector invalid, so the whole rule is dropped	Remove the last comma	D2-08 bug 4
Nothing happens on hover	`a:hoverr` typo; the whole rule is dropped	`a:hover`	D2-08 bug 5
The primary button is not styled	`.btn .primary` (space) means "inside"	`.btn.primary`	D2-FF5 bug 10
`.card` does not match	HTML `class="card, special"` (comma)	`class="card special"`	D2-FF5 bug 11
Your class colour is ignored	An `#id` rule on the same element wins	Use the id rule, or drop it	D2-FF3 (specificity duel)

Check yourself

1. Write the selector for: every `li`; everything with class `veg`; the element with id `cart`.
2. What is the difference between `.btn .primary` and `.btn.primary`? Draw the HTML each one matches.
3. Why does `h1, h2, { color: red; }` style nothing at all?
4. A `<p class="note" id="only">` has three rules: `p` blue, `.note` green, `#only` red. What colour is it? Does the order of the rules matter?
5. Why should you style mostly with classes, not ids?
6. `class="card, special"`: how many classes does this element have, and what are they?
7. Explain class vs id to a friend using "red team" and "Aarav, by name".

Try it

- **D2-08** Selector Mix-ups (5 bugs on the Momo House menu).
- **D2-09** Selector target practice: 5 targets, one rule each, **no HTML edits**.
- **D2-FF2** Hover menu. **D2-FF3** Specificity duel: predict 8 colours, check in DevTools.
- **D2-FF5** Lost & Found: bugs 10–11.

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. A good prompt: *"Here is one HTML element and three CSS rules that match it. Explain, step by step, which colour wins and why. Don't give me new CSS."*

Go deeper

- MDN: CSS selectors (https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Basic_selectors)
- CSS Diner (Wi-Fi): flukeout.github.io: a selector game. Do levels 1–14.

---

*Remember the One Rule: **You may only ship code you can explain, line by line.*** *Next note: 06 · Colours, units, fonts: the paint shop and the measuring tape.*

06 · Colours, Units, Fonts and Text: The Paint Shop and the Measuring Tape

File: notes/06-colours-units-fonts.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2 · Day 2 · Block B6 **Layer:** Frontend (CSS)

Today in one sentence: CSS values are exact: a colour must be a real paint code, and a size needs a number **and** a unit, with no space between them.

The picture to keep in your head

Paint shop (colours), then **measuring tape** (units). Two pictures, on two different slides.

The paint shop. At a paint shop in Kathmandu you can ask for paint in different ways: by **name** ("maroon"), by the **code** on the tin (`#800000`), or by **mixing** red, green and blue yourself (`rgb(128, 0, 0)`). You can also ask for a **see-through** version (`rgba(128, 0, 0, 0.5)`). If you write a code that does not exist (`#gg0000`), the shop gives you nothing, and the wall stays the old colour.

The measuring tape. A tailor never writes "the sleeve is 30". 30 what? Centimetres? Inches? In CSS, a size needs a **unit**: `30px`, `2rem`, `50%`. The tape has different scales:

- `px`: a fixed mark on the tape,
- `%`: "half of the cloth" (a part of the parent),
- `rem`: "times the standard size" (the page's root font size, usually 16 px),
- `em`: "times this element's own font size" (for `font-size` itself: the parent's). `rem` is simpler: × 16 px. Use rem.

Words you now own

Term	What it means (plain English)	Example
Named colour	One of about 140 colour words CSS knows.	`maroon`, `navy`, `white`, `tomato`
Hex colour	`#` + 6 hex digits: 2 for red, 2 for green, 2 for blue. Digits are 0–9 and a–f.	`#14213D` (TSI navy), `#990011` (cherry)
Short hex	3 digits, each one doubled.	`#fff` = `#ffffff`
`rgb()`	Red, green, blue, each 0–255. **Three** numbers.	`rgb(20, 33, 61)`
`rgba()`	rgb + **alpha** (see-through): 0 = invisible, 1 = solid.	`rgba(0, 0, 0, 0.5)`
Unit	What a number measures. **No space** between the number and the unit.	`16px`, `1.5rem`
`px`	Pixels: a fixed size on screen.	`width: 300px;`
`%`	A percentage of the **parent** (for widths).	`width: 50%;`
`rem`	Times the **root** (html) font size, usually 16 px. `2rem` = 32 px.	`font-size: 1.25rem;`
`em`	Times **this element's** font size (for `font-size` itself, × the **parent's**). `rem` is simpler: × 16 px. Use rem.	`padding: 0.5em;`
Font stack	A list of fonts. The browser uses the **first one it has**. End with a generic family.	`"Momo Sans", Arial, sans-serif`
Generic family	A fallback that always exists.	`serif`, `sans-serif`, `monospace`
`line-height`	The height of each line. Best written **without a unit**.	`line-height: 1.6;`

How it works

Colours: four ways to ask for the same paint

```
h1 {
  color: maroon;
}

h2 {
  color: #800000;
}

h3 {
  color: rgb(128, 0, 0);
}

h4 {
  color: rgba(128, 0, 0, 0.5);
}
```

The first three are the **same** colour. The fourth is the same colour at half strength.

How a hex code works:

```
#  99   00   11
   │    │    └─ blue   (00 = none … ff = full)
   │    └────── green
   └─────────── red     → #990011 = lots of red, no green, a little blue = TSI cherry
```

Hex digits are `0 1 2 3 4 5 6 7 8 9 a b c d e f`. There is no `g`. A hex colour has **3 or 6** digits (also 4 or 8 with see-through). `#fffff` (5 digits) is not a colour; it is dropped.

`color` sets the **text** colour. `background-color` sets the **background**. CSS uses the American spelling `color` everywhere in code, even though these notes write "colour" in sentences.

Contrast tip (accessibility): dark text on a light background (or the opposite). TSI navy `#14213D` on white is excellent. Light grey on white fails Lighthouse.

Units: number + unit, glued together

```
width: 300px;       ✔
width: 300;         ✗  no unit → the browser drops the declaration
font-size: 1.5rem;  ✔
font-size: 1.5 rem; ✗  a space breaks it → dropped
width: 0;           ✔   zero is the only length that needs no unit
```

Which unit when?

Use	Good unit	Why
Font sizes	`rem`	Respects the user's browser font setting; easy maths (16 px × n)
Borders, small fixed details	`px`	A 1 px line should stay 1 px
Widths of boxes	`%`, or `px` with `max-width`	Fits different screens
Spacing around text (padding)	`rem` or `em`	Grows with the text
`line-height`	no unit (`1.6`)	Means "1.6 × this element's font size"

rem in one line: the root (`<html>`) font size is 16 px in most browsers, so `1rem` = 16 px, `1.5rem` = 24 px, `2rem` = 32 px.

em in one line: `em` = × this element's font size (for `font-size` itself, × the parent's). A heading with `font-size: 40px` and `padding: 0.5em` gets 20 px of padding. Inside a box with `font-size: 20px`, `font-size: 2em` = 40 px. Nested `em`s multiply, which surprises people. `rem` is simpler: × 16 px. Use rem.

Fonts: always carry a spare

`font-family` takes a **list**. The browser walks the list from left to right and uses the first font installed on the visitor's computer:

```
font-family: "Momo Sans", Arial, Helvetica, sans-serif;
             │            │      │         └─ last resort: any sans-serif font. Always exists.
             │            │      └─ on Mac
             │            └─ on almost every Windows PC
             └─ the font you dream of (here: a font that does not exist anywhere)
```

- Quote names with spaces: `"Times New Roman"`, `"Momo Sans"`.
- Generic families (`serif`, `sans-serif`, `monospace`) are keywords: **never** quote them.
- With **no** fallback, a missing font falls back to the browser's default, which is usually Times New Roman. Your modern menu suddenly looks like an old newspaper.

Text properties you will use every day

Property	Does	Example
`font-size`	size of the letters	`1.125rem`
`font-weight`	thickness	`normal`, `bold`, `700`
`font-style`	italic or not	`italic`
`line-height`	space between lines	`1.6`
`text-align`	left, center, right	`center` (US spelling!)
`text-decoration`	underline or not	`none` (removes link underlines)
`letter-spacing`	space between letters	`2px`
`text-transform`	capital letters	`uppercase`

`line-height: 1.5` and `line-height: 1.5px` are very different: the first means "1.5 lines", the second means "lines 1.5 pixels tall", so every line sits on top of the one before.

Code you built today

Step 1: the paint shop

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TSI Canteen | Paint shop</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>TSI Canteen colours</h1>
  <p class="named">Named: maroon</p>
  <p class="hex">Hex: #990011 (TSI cherry)</p>
  <p class="rgb">rgb: rgb(20, 33, 61) (TSI navy)</p>
  <p class="see-through">rgba: half see-through navy background</p>
</body>
</html>
```

File: `css/style.css`

```
body {
  background-color: #fdf6ec;
}

.named {
  color: maroon;
}

.hex {
  color: #990011;
}

.rgb {
  color: rgb(20, 33, 61);
}

.see-through {
  background-color: rgba(20, 33, 61, 0.3);
}
```

Predict: change `.hex` to `color: #gg0011;`. What colour is that line now? What does DevTools Styles show next to it?

Step 2: the measuring tape

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TSI Canteen | Measuring tape</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Price board sizes</h1>
  <div class="board-px">300px wide, always</div>
  <div class="board-percent">50% of the page width</div>
  <p class="big">2rem text = 32 px</p>
  <p class="small">0.875rem text = 14 px</p>
</body>
</html>
```

File: `css/style.css`

```
.board-px {
  width: 300px;
  background-color: #fca311;
}

.board-percent {
  width: 50%;
  background-color: #14213D;
  color: white;
}

.big {
  font-size: 2rem;
}

.small {
  font-size: 0.875rem;
}
```

Predict: make the browser window narrower. Which board changes width, and which one stays the same?

Step 3: fonts and text on the canteen menu

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TSI Canteen | Menu</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>TSI Canteen menu</h1>
  <p class="tagline">Fresh every morning, Sunday to Thursday</p>
  <p>Didi makes everything in the canteen kitchen. Lunch is served from 12:00 to 14:00. Ask for extra chutney: it is
free.</p>
  <p class="price">Veg momo: Rs. 120 · Chicken momo: Rs. 150 · Milk tea: Rs. 30</p>
</body>
</html>
```

File: `css/style.css`

```
body {
  font-family: "Momo Sans", Arial, Helvetica, sans-serif;
  font-size: 1rem;
  line-height: 1.6;
  color: #14213D;
  background-color: #fdf6ec;
}

h1 {
  font-size: 2rem;
  text-align: center;
  color: #990011;
}

.tagline {
  text-align: center;
  text-transform: uppercase;
  letter-spacing: 2px;
  font-weight: bold;
}

.price {
  font-size: 1.25rem;
  font-style: italic;
}
```

- "Momo Sans" does not exist anywhere, so the browser moves on to Arial. That is the fallback working.
- `line-height: 1.6` (no unit) gives comfortable space between the lines of the long paragraph.

Predict: delete `Arial, Helvetica, sans-serif` so only `"Momo Sans"` is left. What does the text look like? Then change `line-height: 1.6;` to `line-height: 1.6px;`.

Quick reference

Want	Write
Text colour	`color: #14213D;`
Background	`background-color: #fdf6ec;`
See-through	`rgba(20, 33, 61, 0.5)`
Font with fallback	`font-family: "Momo Sans", Arial, sans-serif;`
Readable body text	`font-size: 1rem; line-height: 1.6;`
Big heading	`font-size: 2rem;`
Centre text	`text-align: center;`
No underline	`text-decoration: none;`
Spaced capitals	`text-transform: uppercase; letter-spacing: 2px;`

TSI palette used in class: navy `#14213D`, cherry `#990011`, amber `#fca311`, cream `#fdf6ec`.

Mistakes everyone makes

You see…	Because…	Fix	Planted in
The box ignores its width	`width: 300;` no unit	`300px`	D2-10 bug 1, D2-HW1 bug 7 (`padding: 20;`), D2-FF4 (`padding: 20`)
The colour does not change	`#gg0000`: g is not a hex digit	`#cc0000` (0–9, a–f only)	D2-10 bug 2
The colour does not change	`#fffff`: 5 digits	`#ffffff` or `#fff`	D2-HW1 bug 11
The size does not change	`1.5 rem` with a space	`1.5rem`	D2-10 bug 3, D2-FF4 (`16 px`), D2-FF5 bug 8 (`20 px`)
The page looks like an old newspaper	`"Momo Sans"` with no fallback	`"Momo Sans", Arial, sans-serif`	D2-10 bug 4
The lines sit on top of each other	`line-height: 1.5px`	`line-height: 1.5`	D2-10 bug 5
The background stays white	`rgb(255, 200)`: only two numbers	three numbers: `rgb(255, 200, 0)`	D2-HW4 bug 7
The page scrolls sideways	`.banner { width: 120%; }` is wider than its parent	`width: 100%` (or less)	D2-HW4 bug 8
DevTools: "Unknown property name"	`colour:` British spelling in code	`color:`	D2-06 bug 5, D2-FF4

Check yourself

1. Write TSI navy three ways: hex, rgb, and rgba at 50 %.
2. Why is `#gg0000` not a colour? Which characters can a hex code use?
3. What is `1.5rem` in pixels, if the root font size is 16 px? And `0.75rem`?
4. What is wrong with `font-size: 1.5 rem;` and with `width: 300;`?
5. Why does a font stack end with `sans-serif` or `serif`, and why is that word not in quotes?
6. What is the difference between `line-height: 1.5` and `line-height: 1.5px`?
7. Explain `rem` vs `%` to a friend using the measuring tape.

Try it

- **D2-10** Units & Colours (5 bugs on the canteen price page).
- **D2-11** Theme the canteen menu: a 3-colour hex palette, a font stack with a fallback, rem sizes, `line-height: 1.6`.
- **D2-FF1** TSI ID card (fonts and colours from a picture).
- **D2-HW4** Patan Craft Shop replay: bugs 6–8.

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. A good prompt: *"Explain the difference between px, %, rem and em with one example each about font size and box width. Which should a beginner use for font sizes, and why? No full stylesheet, please."*

Go deeper

- MDN: CSS values and units (https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Values_and_units)
- MDN: Fundamental text and font styling (https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Text_styling/Fundamentals)

*Remember the One Rule: **You may only ship code you can explain, line by line.*** *Next note: 07 · The box model: parcels being shipped.*

07 · The Box Model: Parcels Being Shipped

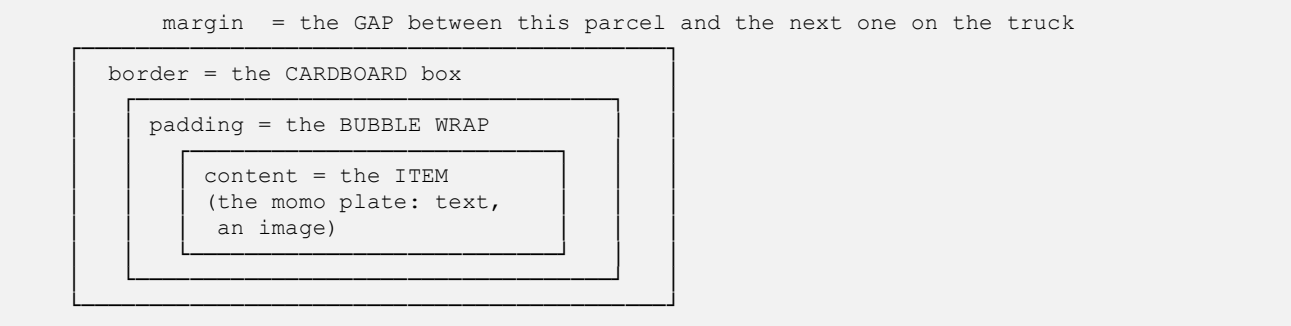
File: notes/07-box-model.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2 · Day 2 · Block B7 **Layer:** Frontend (CSS)

Today in one sentence: every element on a web page is a box made of four layers (content, padding, border, margin), and you can measure every layer.

The picture to keep in your head

Boxes being shipped (parcels being shipped). Didi sends a plate of momo to a customer by courier:



Parcel	CSS	Has a colour?
The item	**content** (`width`, `height`)	the background colour shows here
Bubble wrap	**padding**	yes: the background colour fills the padding too
Cardboard	**border**	its own colour (`border: 2px solid navy`)
Gap to the next parcel	**margin**	no: always see-through

More bubble wrap makes the parcel **bigger**, even if the item stays the same. That is the most important idea today, and the cause of the most common bug (see box-sizing below).

Words you now own

Term	What it means (plain English)	Example
Box model	The rule that every element is a box of content + padding + border + margin.	everything on the page
`width` / `height`	The size of the **content** (by default).	`width: 300px;`
`padding`	Space **inside** the border, around the content. Cannot be negative.	`padding: 20px;`
`border`	The line around the padding: **width, style, colour**.	`border: 2px solid #14213D;`
`margin`	Space **outside** the border, between boxes. Can be negative; can be `auto`.	`margin: 16px;`
Shorthand	One property that sets several sides at once.	`padding: 10px 20px;`
`margin: 0 auto`	No margin top and bottom; left and right share the free space equally → the box is **centred**. Needs a width.	`.wrapper { max-width: 960px; margin: 0 auto; }`
`box-sizing: border-box`	Makes `width` include the padding and the border.	`* { box-sizing: border-box; }`
`border-radius`	Rounds the corners. `50%` on a square = a circle.	`border-radius: 12px;`
`max-width`	"Be at most this wide; narrower on small screens is fine."	`max-width: 960px;`
Box diagram	The coloured rectangles in DevTools (Elements → Computed) showing each layer in pixels.	margin orange, border yellow, padding green, content blue

How it works

The box maths

By default (`box-sizing: content-box`), `width` is only the **item**. The visible box is bigger:

```
visible width = border-left + padding-left + width + padding-right + border-right

.card { width: 300px; padding: 20px; border: 5px solid navy; }

          5   +   20   +  300  +   20   +   5   = 350px on screen   (not 300!)
       border  padding  content padding border
```

Three of those cards side by side need 3 × 350 = **1050 px**. In a 960 px row, the third card does not fit, so it **wraps** to the next line. (The cards are `display: inline-block`, see note 08. Normal block boxes never sit side by side.)

box-sizing: border-box (the fix)

```
.card { box-sizing: border-box; width: 300px; padding: 20px; border: 5px solid navy; }

the whole parcel is 300px; the item shrinks to 300 − 20 − 20 − 5 − 5 = 250px
three cards = 900px → they fit in a 960px row
```

Most developers put this at the top of every stylesheet, so widths always mean "the whole parcel":

```
* {
  box-sizing: border-box;
}
```

(`*` is the **universal** selector: it matches every element.)

Shorthand: 1, 2, 3 or 4 values

Padding and margin go **clockwise from the top**, like a clock: **T**op, **R**ight, **B**ottom, **L**eft. Remember **"TRouBLe"**.

You write	Means
`padding: 10px;`	all four sides 10 px
`padding: 10px 20px;`	top + bottom 10 px · left + right 20 px
`padding: 10px 20px 30px;`	top 10 · left + right 20 · bottom 30
`padding: 10px 20px 30px 40px;`	top 10 · right 20 · bottom 30 · left 40
`padding: 10px 20px 10px 20px 5px;`	**5 values = invalid** → the whole declaration is dropped

Single sides also exist: `padding-top`, `margin-left`, `border-bottom`…

Border: three parts, and the style is not optional

```
border:  2px     solid    #fca311;
         width   STYLE    colour
```

The style can be `solid`, `dashed`, `dotted`, `double`… The default style is `none`. So `border: 2px orange;` is accepted, but the style stays `none`, and **no border appears**. (Chrome keeps the declaration; the border is just invisible.)

Centring a box: margin: 0 auto

`margin: 0 auto` says: "top and bottom 0; left and right `auto` (share the free space)". It only works when the box is **narrower** than its parent. A block box with no width already fills the whole line, so there is no free space to share, and nothing moves. Give it a `width` or (better) a `max-width`.

Negative values

- `padding: -10px;` is **invalid** (you can't have negative bubble wrap). Chrome drops it, so the padding stays 0.
- `margin: -10px;` is **valid**: it pulls the box closer (or on top of a neighbour). Use it rarely.

Rounded corners and round photos

`border-radius: 12px` rounds the corners. `border-radius: 50%` on a **square** element makes a **circle**. On a rectangle (120 × 80), `50%` makes an **oval**. For a round avatar, make the width and the height equal.

Height and long text

If you fix `height: 100px` and the text needs 160 px, the text does not get cut: it **spills out** of the box and runs over the next thing. Usually it is better not to set a height at all; let the content decide.

See it in DevTools

Right-click the card → **Inspect** → **Computed** tab (or the bottom of the Styles tab). The box diagram shows margin, border, padding and content with their real pixel numbers. Hover over each layer: the page highlights it.

Code you built today

Step 1: content width and padding

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Box model | Step 1</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>TSI student cards</h1>
  <div class="card">
    <h2>Sunita Gurung</h2>
    <p>Morning batch · Dillibazar</p>
  </div>
</body>
</html>
```

File: `css/style.css`

```
.card {
  width: 300px;
  padding: 20px;
  background-color: #fdf6ec;
}
```

Predict: how wide is the cream box on screen? Check with F12 → Computed.

Step 2: border and margin

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Box model | Step 2</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>TSI student cards</h1>
  <div class="card">
    <h2>Sunita Gurung</h2>
    <p>Morning batch · Dillibazar</p>
  </div>
  <div class="card">
    <h2>Aarav</h2>
    <p>Evening batch · Dillibazar</p>
  </div>
</body>
</html>
```

File: `css/style.css`

```
.card {
  width: 300px;
  padding: 20px;
  border: 5px solid #14213D;
  margin: 16px;
  background-color: #fdf6ec;
}
```

- Each card is now 5 + 20 + 300 + 20 + 5 = **350 px** wide.
- The 16 px margin is the gap between the two parcels.

Predict: change `border: 5px solid #14213D;` to `border: 5px #14213D;`. What happens to the border?

Step 3: centre it and fix the maths

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Box model | Step 3</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <div class="wrapper">
    <h1>TSI student cards</h1>
    <div class="card">
      <h2>Sunita Gurung</h2>
      <p>Morning batch · Dillibazar</p>
    </div>
  </div>
</body>
</html>
```

File: `css/style.css`

```
* {
  box-sizing: border-box;
}

.wrapper {
  max-width: 600px;
  margin: 0 auto;
  padding: 24px;
  background-color: #fdf6ec;
}

.card {
  width: 300px;
  margin: 0 auto;
  padding: 20px;
  border: 5px solid #14213D;
  background-color: white;
}
```

- `box-sizing: border-box`: the card is now exactly **300 px** on screen.
- The wrapper is at most 600 px wide and centred on the page. The card is centred inside the wrapper.

Predict: delete `max-width: 600px;` from `.wrapper`. Does the cream wrapper still look centred? Why?

Step 4: the profile card (D2-13)

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sunita's profile card</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <main class="profile">
    <img class="avatar" src="images/avatar-sunita.svg" alt="Sunita Gurung" width="120" height="120">
    <h1>Sunita Gurung</h1>
    <p>Student, AI-Powered Full Stack Developer Program</p>
    <p>TSI · Dillibazar, Kathmandu</p>
  </main>
</body>
</html>
```

File: `css/style.css`

```
* {
  box-sizing: border-box;
}

body {
  background-color: #fdf6ec;
  font-family: Arial, Helvetica, sans-serif;
}

.profile {
  width: 320px;
  margin: 40px auto;
  padding: 24px;
  border: 2px solid #14213D;
  border-radius: 12px;
  background-color: white;
  text-align: center;
}

.avatar {
  width: 120px;
  height: 120px;
  border: 4px solid #990011;
  border-radius: 50%;
}
```

```
h1 {
  font-size: 1.5rem;
  color: #14213D;
}
```

- `margin: 40px auto`: 40 px top and bottom, centred left and right.
- The avatar is 120 × 120 (a square), so `border-radius: 50%` gives a **circle** with a cherry ring.

Predict: change the avatar's `height` to `80px`. What shape do you get?

Quick reference

Want	Write
Widths that include padding + border	`* { box-sizing: border-box; }`
Space inside	`padding: 20px;` / `padding: 10px 20px;`
Space outside	`margin: 16px;`
A line around	`border: 2px solid #14213D;` (width style colour)
Centre a box	`max-width: 960px; margin: 0 auto;`
Round corners	`border-radius: 12px;`
A circle	equal width and height + `border-radius: 50%;`
Shorthand order	top right bottom left (TRouBLe)

Mistakes everyone makes

You see…	Because…	Fix	Planted in
The third card wraps to a new line	width + padding + border = 350 px, not 300	`box-sizing: border-box`	D2-12 bug 1, D2-HW1 bug 9
`margin: 0 auto` does not centre	The box has no `width`/`max-width`, so it fills the line	Add `max-width`	D2-12 bug 2
The padding is 0	`padding: -10px;` is invalid and dropped	Use 0 or a positive value	D2-12 bug 3
No border at all	`border: 2px orange;` has no style	`border: 2px solid orange;`	D2-12 bug 4, D2-HW1 bug 8, D2-FF4
An oval avatar	`border-radius: 50%` on a 120 × 80 image	Equal width and height	D2-12 bug 5
The padding is ignored	`padding: 10px 20px 10px 20px 5px;` (5 values)	1 to 4 values only	D2-HW4 bug 6
Text spills out of the box	`.card { height: 100px; }` with long text	Remove the fixed height	D2-HW4 bug 9
`padding: 20` does nothing	no unit	`20px`	D2-HW1 bug 7, D2-FF4

Check yourself

1. Name the four layers of the box model, from the inside out, with the parcel word for each.
2. A box has `width: 200px; padding: 10px; border: 2px solid;`. How wide is it on screen? And with `box-sizing: border-box`?
3. What does `padding: 5px 15px;` mean, side by side?
4. Why does `border: 2px orange;` show no border?
5. `margin: 0 auto` does nothing. What is missing?
6. Which can be negative: padding or margin?
7. How do you make a round photo, and why can it come out oval?
8. Explain padding vs margin to a friend using the parcel and the bubble wrap.

Try it

- **D2-12** Broken Cards (5 bugs: Aarav, Sunita and Rajan's cards).
- **D2-13** Profile card: then show & tell.
- **D2-FF1** TSI ID card: rebuild it from `target.svg` (card 360 × 220, padding 16 px, 12 px radius, 80 px round avatar).
- **D2-15** Style Momo House, checkpoint 4: 3 dish cards.

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. A good prompt: *"My three cards are each 300px wide in a 960px row, but the third one wraps to the next line. Here is my CSS. Walk me through the box-model maths. Don't give me the fixed code."*

Go deeper

- MDN: The box model (https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Box_model)
- Chrome DevTools docs: "View and change CSS" → the box model diagram.

---

*Remember the One Rule: **You may only ship code you can explain, line by line.*** *Next note: 08 · Display: the queue vs the bench.*

08 · Display: The Queue vs the Bench

File: notes/08-display.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2 · Day 2 · Block B8 **Layer:** Frontend (CSS)

Today in one sentence: `display` decides whether a box stands in its own line, sits next to others, or leaves the page.

The picture to keep in your head

The queue vs the bench. Think of the TSI Canteen at lunch time:

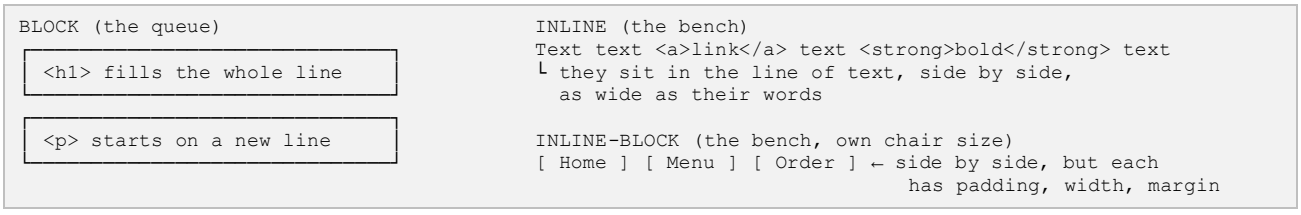
Canteen	`display` value	How it behaves
The queue: each student stands on their **own line**, one behind the other, as wide as the queue	`block`	starts on a new line, fills the full width; width, height, margin and padding all work
The bench: students sit **side by side**; you **can't push one person up or down**	`inline`	sits in the line of text, as wide as its content; `width`, `height` and vertical margin are **ignored**
Side by side on the bench, but each person **keeps their own size** (a chair you can resize)	`inline-block`	sits in the line like inline, but width, height, margin and padding work like block
The student **left the room**	`none`	gone: takes no space at all
(bonus) The student is **invisible but still sitting there**	`visibility: hidden`	not seen, but the empty seat still takes its space

Words you now own

Term	What it means (plain English)	Example elements
`display`	The CSS property that says how a box joins the layout.	`display: inline-block;`
Block	Own line, full width.	`div`, `p`, `h1`–`h6`, `ul`, `li`, `section`, `header`, `nav`, `footer`
Inline	In the line of text, content width.	`a`, `span`, `strong`, `em`, `img` (an inline element with a size of its own)
Inline-block	In the line, but with its own width, height and vertical margins.	buttons, nav links, badges
`none`	Removed from the page (still in the HTML).	a hidden logo
`visibility: hidden`	Invisible, but keeps its space.	an empty gap where a badge was
Default display	Every element has one before you write any CSS.	`li` is block → a vertical list
`list-style: none`	Removes the bullets (or numbers) of a list.	`ul.menu { list-style: none; }`
Horizontal nav	A row of links made from a `ul` of `li`s set to `inline-block`.	Home · Menu · Order · About

How it works

Block vs inline, in one picture



Why inline ignores vertical margin

An inline box lives **inside a line of text**. If it could push itself down with `margin-top`, it would break the line. So the browser **ignores** vertical margin on inline boxes, and `width` / `height` too.

Vertical **padding** on an inline box is even trickier: the background grows up and down, but the line does **not** move. So the coloured link **overlaps** the lines above and below it. We checked this in Chrome: a link with `display: inline; padding: 12px 16px; margin-top: 20px;` stays exactly in its line (margin-top ignored), and its 41 px tall background spills over its neighbours.

The fix for nav links is almost always `display: inline-block`: now padding and margin push things away properly.

none vs visibility: hidden

```
display: none        → [A] [C]        B left the room. A and C close the gap.
visibility: hidden   → [A] [  ] [C]   B is invisible, but its seat is still taken.
```

`display: none` is how you hide something completely. `visibility: hidden` is rare; use it only when you really want the empty space.

The recipe for a horizontal nav

A menu is a **list of links**, so the HTML is a `ul`. Then three lines of CSS make it horizontal:

```
1. ul.menu      → list-style: none;   (no bullets)  + padding: 0; (no indent)
2. .menu li     → display: inline-block;   (side by side, keep their size)
3. .menu a      → display: inline-block; padding: …;   (big, clickable buttons)
```

You will learn flexbox in session 3.2, which is the modern way to do this. Today's inline-block way still works everywhere, and it teaches you how display really behaves.

Code you built today

Step 1: block and inline side by side

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Display | Queue and bench</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>The queue and the bench</h1>
  <p>This paragraph is a block. It has a <a href="index.html">link</a> and a <span class="tag">tag</span> inside it.</p>
  <p>Another block, on its own line.</p>
  <p>Inline-block badges: <span class="badge">Veg</span> <span class="badge">Spicy</span> <span
class="badge">New</span></p>
</body>
</html>
```

File: `css/style.css`

```
p {
  background-color: #fdf6ec;
}

.tag {
  background-color: #fca311;
  margin-top: 30px;
  width: 200px;
}

.badge {
  display: inline-block;
  width: 70px;
  margin-top: 10px;
  padding: 4px 8px;
  background-color: #14213D;
  color: white;
  text-align: center;
}
```

- Each `p` is a block: own line, full width (look at the cream background).
- The `.tag` span is inline, so its `width: 200px` and `margin-top: 30px` are ignored.
- The badges are inline-block: side by side, each 70 px wide, pushed down by their margin.

Predict: change `.badge` to `display: inline;`. Which of its properties stop working?

Step 2: the horizontal Momo House nav

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House | Nav</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header class="site-header">
    <img class="logo" src="images/logo-momo-house.svg" alt="Momo House logo" width="60">
    <nav>
      <ul class="menu">
        <li><a href="index.html">Home</a></li>
        <li><a href="index.html">Menu</a></li>
        <li><a href="index.html">Order</a></li>
        <li><a href="index.html">About</a></li>
      </ul>
    </nav>
  </header>
  <main>
    <h1>Fresh momo in Thamel</h1>
  </main>
</body>
</html>
```

File: `css/style.css`

```
.site-header {
  background-color: #14213D;
  padding: 12px 24px;
}

.menu {
  list-style: none;
  padding: 0;
  margin: 0;
}

.menu li {
  display: inline-block;
}

.menu a {
  display: inline-block;
  padding: 12px 16px;
  color: white;
  text-decoration: none;
}

.menu a:hover {
  background-color: #990011;
}
```

- `list-style: none` removes the bullets. `padding: 0` removes the list's indent.
- `.menu li { display: inline-block; }` puts the four items side by side.
- The links are inline-block too, so their padding makes big, safe click areas that don't overlap anything.

Predict: delete the `.menu li` rule. What happens to the menu? Then put it back and change `.menu a` to `display: inline;`. What changes on hover?

Step 3: none vs visibility

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Display | None vs hidden</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Who left the room?</h1>
  <p class="row"><span class="seat">Aarav</span><span class="seat gone">Sunita</span><span class="seat">Rajan</span></p>
  <p class="row"><span class="seat">Aarav</span><span class="seat hidden">Sunita</span><span
class="seat">Rajan</span></p>
</body>
</html>
```

File: `css/style.css`

```
.seat {
  display: inline-block;
  width: 100px;
  margin-right: 8px;
  padding: 8px;
  background-color: #fca311;
  text-align: center;
}

.gone {
  display: none;
}

.hidden {
  visibility: hidden;
}
```

Predict: in which row is there a gap between Aarav and Rajan? Why?

Quick reference

Want	Write
Own line, full width	`display: block;`
Side by side, sizes work	`display: inline-block;`
Gone completely	`display: none;`
Invisible, keep the space	`visibility: hidden;`
No bullets	`list-style: none;` (+ `padding: 0;`)
Horizontal nav	`ul { list-style: none; }` · `li { display: inline-block; }` · `a { display: inline-block; padding: …; }`

Element	Default display
div, p, h1–h6, ul, ol, li, section, header, nav, main, footer, form, table	block
a, span, strong, em, label, img, input, button	inline (img, input and button behave like inline-block)

Mistakes everyone makes

You see…	Because…	Fix	Planted in
Nothing changed	`display: blok;` typo; dropped	`display: block;`	D2-14 bug 1, D2-FF4
The logo is missing	`.logo { display: none; }` left in	Delete it	D2-14 bug 2
Bullets in the nav	`list-style: none` missing	Add it on the `ul`	D2-14 bug 3
Nav links overlap the hero; `margin-top` does nothing	Links are `display: inline`	`display: inline-block`	D2-14 bug 4
The nav stays vertical	CSS `.menu-li`, HTML `<ul class="menu"><li>`	`.menu li`	D2-14 bug 5
Nav stays vertical	`display: inline-blok;` typo	`inline-block`	D2-HW1 bug 5
An empty gap where the badge was	`visibility: hidden` instead of `display: none`	`display: none`	D2-HW4 bug 10

Check yourself

1. Name four block elements and four inline elements.
2. Which three properties does an inline element ignore?
3. What is the difference between `inline` and `inline-block`?
4. `display: none` vs `visibility: hidden`: which one leaves a gap?
5. Write the three rules that turn a `ul` of links into a horizontal nav.
6. A nav link has `margin-top: 20px` but does not move. Why? How do you fix it?
7. Explain block vs inline to a friend using the queue and the bench.

Try it

- **D2-14** Nav Won't Behave (5 bugs on the Momo House header).
- **D2-15** Style Momo House, checkpoint 2: header + horizontal nav.
- **D2-FF2** Hover menu.
- **D2-HW1** bug 5 and **D2-HW4** bug 10.

Coming later

Flexbox and grid (session 3.2) are the modern tools for rows and columns. `position` (relative, absolute, fixed, sticky) and responsive design with media queries also come later. Today's inline-block nav is the foundation.

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. A good prompt: *"Why does margin-top do nothing on my nav links, while padding makes the background overlap the next section? Explain inline vs inline-block. Don't write my CSS."*

Go deeper

- MDN: display (https://developer.mozilla.org/en-US/docs/Web/CSS/display)
- MDN: Block and inline layout in normal flow (https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_display/Block_and_inline_layout_in_normal_flow)

---

*Remember the One Rule: **You may only ship code you can explain, line by line.*** *Next note: 09 · Debugging CSS: what the browser drops.*

09 · Debugging CSS: What the Browser Drops

File: notes/09-debugging-css.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2 · Day 2 · All CSS blocks (B4–B9) **Layer:** Frontend (CSS)

Today in one sentence: CSS never shows an error message; when the browser does not understand something, it quietly **drops** it, so your job is to find what was dropped and why.

The picture to keep in your head

On Day 1 you learnt that browsers are **polite guessers**: they never show an HTML error, they guess. With CSS the browser is polite in a different way: it does not guess, it **skips**. Anything it cannot read, it drops, and it carries on with the next thing it *can* read.

Think of the **school uniform** list (note 04) being read aloud by a strict school guard. If one line says "shirt: bloo", the guard does not guess "blue". He skips that line and reads the next one. If a line has no full stop at the end, he reads two lines as one long nonsense line and skips both. If a bracket is never closed, he thinks every line after it still belongs to the first item.

So debugging CSS means asking four questions, in this order:

```
1. Did the stylesheet LOAD at all?            → F12 → Network (404?) and the <link> tag
2. Does the selector MATCH the element?       → Inspect the element: is the rule listed in Styles?
3. Did the browser UNDERSTAND the declaration? → a yellow ⚠ triangle = "Invalid property value" / "Unknown property name"
4. Did another rule WIN?                       → crossed out with no ⚠ = a later or stronger rule won
```

Words you now own

Term	What it means (plain English)	Example
Drop	The browser ignores something it cannot read, silently.	`width: 300;` → dropped
Invalid declaration	A `property: value` the browser cannot use (bad value, bad unit, unknown property). Only **that declaration** is dropped.	`color: #gg0000;`
Invalid selector	A selector the browser cannot read. The **whole rule** is dropped.	`h1, h2, { … }`
Parse	How the browser reads your text, symbol by symbol (`{`, `:`, `;`, `}`).	
⚠ warning triangle	DevTools' mark for a declaration it did not understand.	"Invalid property value"
Crossed out	DevTools' mark for a declaration that lost the cascade.	a later `h1` rule won
404	"Not found": the browser asked for a file that is not there.	`css/styles.css`
Network panel	F12 → Network: every file the page asked for, with its status.	`style.css 200`

How it works

The four kinds of drop

1. **A declaration is dropped** (the smallest drop). A bad value or an unknown property: just that one `property: value;` is ignored. The other lines in the rule still work.
2. **A whole rule is dropped**. If the **selector** is invalid (a trailing comma, an unknown `:state`, a `//` before it), the browser throws away the whole `{ … }` block.
3. **The next rule is swallowed**. A missing `}` means the rule never ends, so the browser reads the next rule as if it were **inside** the first one. It no longer works on its own.
4. **The whole file is missing**. A bad `rel` or a wrong `href`: no CSS loads at all, and every other bug is hidden. **Always fix this first.**

The table: what the browser drops

Every row below was tested in Chrome (we loaded a test page and read the computed styles back). "Kept" means Chrome used it.

You wrote (broken)	What Chrome does	What you see	Fix	Planted in
`colour: maroon;`	**Unknown property → declaration dropped.** The next declaration still works.	Text stays black	`color: maroon;`	D2-06 bug 5, D2-FF4
`color: #gg0000;`	**Invalid value → declaration dropped** (g is not hex). Other declarations in the rule still apply.	Old colour stays	`#cc0000`	D2-10 bug 2
`color: #fffff;` (5 digits)	Invalid value → dropped	Old colour stays	`#ffffff`	D2-HW1 bug 11
`width: 300;`	No unit → dropped	Box stays full width	`300px`	D2-10 bug 1
`padding: 20;`	No unit → dropped	No padding	`20px`	D2-HW1 bug 7, D2-FF4
`font-size: 1.5 rem;` / `20 px`	A space between number and unit → dropped	Size does not change	`1.5rem` / `20px`	D2-10 bug 3, D2-FF4, D2-FF5 bug 8
`padding: -10px;`	Negative padding is invalid → dropped (padding stays 0). *Negative **margin** is kept.*	No padding	`10px`	D2-12 bug 3
`padding: 10px 20px 10px 20px 5px;`	5 values → dropped	No padding	1 to 4 values	D2-HW4 bug 6
`background-color: rgb(255, 200);`	Missing a number → dropped	No background	`rgb(255, 200, 0)`	D2-HW4 bug 7
`display: blok;` / `inline-blok`	Unknown keyword → dropped	Display stays the default	`block` / `inline-block`	D2-14 bug 1, D2-HW1 bug 5, D2-FF4
`background-color = #fdf6ec;`	`=` instead of `:` → that declaration dropped; Chrome recovers at the next `;`	No background	`background-color: #fdf6ec;`	D2-FF5 bug 9
`color: navy background-color: yellow;` (missing `;`)	The two lines become **one** bad value → **both** are dropped. Declarations after the next `;` work again	Neither colour applies	add `;` after `navy`	D2-06 bug 4, D2-FF4
`h1, h2, { color: red; }`	Trailing comma = invalid selector → **whole rule dropped**	Neither h1 nor h2 is red	`h1, h2 { … }`	D2-08 bug 4
`a:hoverr { … }`	Unknown pseudo-class = invalid selector → **whole rule dropped**	Nothing on hover	`a:hover`	D2-08 bug 5
`// footer styles` then `footer { … }`	`//` is not a CSS comment. It becomes part of the next selector, which is now invalid → **the rule after it is dropped**. The rule after *that* works	Footer unstyled	`/* footer styles */`	D2-06 bug 6, D2-FF4
`h1 { color: red;` (missing `}`) then `p { … }`	The `h1` rule never ends, so Chrome reads `p { … }` as a rule **nested inside** it. The `p` rule **no longer works on its own** (it is swallowed)	The next rule does nothing	close the `}`	D2-06 bug 3, D2-HW1 bug 10, D2-FF4
`border: 2px orange;`	**Kept**, but the style defaults to `none` → no border is drawn	No border	`border: 2px solid orange;`	D2-12 bug 4, D2-HW1 bug 8
`line-height: 1.5px;`	**Kept** (it is a valid length) → lines 1.5 px tall	Lines overlap	`line-height: 1.5;`	D2-10 bug 5
`font-family: "Momo Sans";`	**Kept**, but the font does not exist → the browser's default font (Times New Roman)	Old-newspaper look	add `Arial, sans-serif`	D2-10 bug 4
`.card` vs `class="Card"`	**Kept**, but it matches nothing (case-sensitive)	Rule does nothing	same case	D2-08 bug 1, D2-HW1 bug 6
`rel="stylsheet"` / `href="css/styles.css"`	The file is **not loaded at all**	No CSS anywhere	`stylesheet` / the real path	D2-06 bugs 1–2, D2-HW1

You wrote (broken)	What Chrome does	What you see	Fix	Planted in
				bug 1

Notice the last five rows: the browser **kept** them. They are valid CSS that does the wrong thing. DevTools shows no ⚠ triangle for them. For those, you must read the code against what you meant (or use the box diagram).

Where DevTools shows each kind

Kind	What you see in F12
File not loaded	Network: `style.css` in red, **404**. Styles: none of your rules appear at all
Selector does not match	Inspect the element: your rule is **not listed** in Styles
Invalid selector	The rule is **not listed** anywhere (it was thrown away)
Invalid declaration	Listed, crossed out, with a yellow ⚠ ("Invalid property value" or "Unknown property name")
Declaration lost the cascade	Listed, crossed out, **no** ⚠; the winner is higher up in the Styles list
Valid but wrong (`border: 2px orange`)	Listed, not crossed out. Check the Computed tab: `border-top-style: none`

The repair order

1. **Load it.** Check the `<link>`: `rel="stylesheet"`, the path from the HTML file, the exact file name.
2. **Close everything.** Every `{` has a `}`. Every declaration ends with `;`. VS Code helps: put the cursor on a `{` and its partner `}` is highlighted.
3. **Read the triangles.** Fix every ⚠ in Styles.
4. **Match the names.** Compare each selector with the HTML, letter by letter, case included.
5. **Check the maths.** Box sizes (`box-sizing`), units, number of values.

Code you built today

Step 1: a stylesheet with six silent problems

Read it like the strict guard. Which lines will Chrome skip?

✗ Broken on purpose: find what is wrong

```
body {
  background-color = #fdf6ec;
  font-family: "Momo Sans";
}

h1, h2, {
  colour: #990011;
}

.price {
  font-size: 1.5 rem;
  color: #14213D
  font-weight: bold;
}

// footer styles
footer {
  border: 2px orange;
}
```

Predict: list each problem and say whether Chrome drops the declaration, drops the whole rule, or keeps it but it looks wrong. Then type the page below and the broken CSS, open F12, and check your list against the ⚠ triangles.

Step 2: the same stylesheet, repaired

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House | Debug lab</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Momo House</h1>
  <h2>Today's prices</h2>
  <p class="price">Steam momo: Rs. 150</p>
  <p class="price">Jhol momo: Rs. 200</p>
  <footer>
    <p>Thamel, Kathmandu · Open Sunday to Friday</p>
  </footer>
</body>
</html>
```

File: `css/style.css`

```
body {
  background-color: #fdf6ec;
  font-family: "Momo Sans", Arial, Helvetica, sans-serif;
}

h1,
h2 {
  color: #990011;
}

.price {
  font-size: 1.5rem;
  color: #14213D;
  font-weight: bold;
}

/* footer styles */
footer {
  border: 2px solid orange;
}
```

- `=` → `:` · fallback fonts added · trailing comma removed · `colour` → `color` · `1.5 rem` → `1.5rem` · missing `;` added after `#14213D` · `//` → `/* */` · `solid` added to the border.

Predict: in the broken version, was `font-weight: bold` on `.price` working or not? (Hint: count the `;`.)

Step 3: the missing `}` swallows the next rule

✗ Broken on purpose: find what is wrong

```
h1 {
  color: #990011;

p {
  color: #14213D;
}
```

Chrome reads this as "an `h1` rule with a `p` rule inside it". So the paragraphs are **not** navy. (The `h1` is still cherry.) Add the `}` after `color: #990011;` and both rules work again.

Quick reference: the debugging checklist

```
☐ Network: style.css is 200, not 404
☐ <link rel="stylesheet" href="…"> spelt right, path right from THIS html file
☐ Every { has a }        ☐ Every declaration ends with ;
☐ Comments are /* */     ☐ US spelling: color, center
☐ Numbers have units, no space: 20px, 1.5rem (0 is fine alone)
☐ Hex: # + 3 or 6 digits of 0-9 a-f        ☐ rgb(): three numbers
☐ Selector names = HTML names, same case   ☐ No trailing comma in a selector list
☐ Border has a style (solid)               ☐ Box maths: box-sizing: border-box
☐ No ⚠ triangles left in Styles             ☐ Crossed out without ⚠? A stronger/later rule won
```

Mistakes everyone makes

You see…	Because…	Fix	Planted in
You fix CSS bugs but nothing changes	The stylesheet never loaded	Fix the `<link>` first	D2-06 (README: fix bugs 1 and 2 first), D2-HW1 bug 1
You change the CSS but the page does not update	Not saved, or Live Server not running	Ctrl+S; check the Live Server "Port" button	(every exercise)
One rule is fine, the next is dead	Missing `}`	Close it	D2-06 bug 3
A whole rule is dead, no ⚠	Invalid selector (`h1, h2,` or `a:hoverr`) or a `//` line before it	Fix the selector / comment	D2-08 bugs 4–5, D2-06 bug 6
Two declarations in a row are dead	Missing `;`	Add it	D2-06 bug 4
Everything looks right in CSS, nothing matches	Selector and HTML names differ	Copy-paste the class name to compare	D2-08 bugs 1–3
Ten mixed bugs in one file	All of the above	Follow the repair order	D2-FF4 (10 bugs), D2-HW1 (12 bugs)

Check yourself

1. Name the four questions to ask when a style does not work, in order.
2. What is the difference between a dropped **declaration** and a dropped **rule**? Give one example of each.
3. Why does a missing `;` break two declarations, not one?
4. Why does `// footer styles` break the footer rule but not the rule after it?
5. `border: 2px orange;` is not crossed out in DevTools, but there is no border. Why?
6. Your rule is not listed at all in the Styles pane. Name two possible reasons.
7. Explain "the browser drops what it can't read" to a friend, using the strict school guard.

Try it

- **D2-06** 🐞 Stylesheet Not Loading · **D2-08** 🐞 Selector Mix-ups · **D2-10** 🐞 Units & Colours · **D2-12** 🐞 Broken Cards · **D2-14** 🐞 Nav Won't Behave
- **D2-FF4** 🐞 Haunted stylesheet (10 mixed bugs) · **D2-FF5** 🐞 Lost & Found (11 bugs)
- **D2-HW1** 🐞 Haunted portfolio repair (12 bugs) · **D2-HW4** 🐞 Patan Craft Shop replay (10 bugs, stretch)

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. Try the checklist first, then ask. A good prompt: *"DevTools shows a yellow warning triangle 'Invalid property value' next to `font-size: 1.5 rem;`. Explain what makes this value invalid. Don't fix the rest of my file."*

Go deeper

- MDN: Debugging CSS (https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Debugging_CSS)
- MDN: CSS error handling (https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_syntax/Error_handling)

---

*Remember the One Rule: **You may only ship code you can explain, line by line.*** *Next note: 10 · Bootstrap teaser: IKEA for websites.*

10 · Bootstrap Teaser: IKEA for Websites

File: notes/10-bootstrap-teaser.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2 · Day 2 · Block B10 **Layer:** Frontend (CSS framework)

Today in one sentence: Bootstrap is a big stylesheet someone else wrote; you use its ready-made classes instead of writing every rule yourself.

The picture to keep in your head

IKEA furniture (ready-made parts vs raw wood): **IKEA for websites**.

Today you built furniture from **raw wood**: you cut every board yourself (`padding: 20px`, `border: 2px solid`, `border-radius: 12px`…). It takes time, but you understand every piece.

IKEA gives you **ready-made parts** with names: "BILLY shelf", "LACK table". You don't cut wood. You pick the part and assemble it. Bootstrap does the same for websites: it gives you ready-made **classes** with names: `container`, `btn`, `card`.

IKEA	Bootstrap
The IKEA store (all the parts)	`bootstrap.min.css` (one big stylesheet, about 230 KB)
A part with a name: "BILLY shelf"	a class: `card`, `btn`, `container`
A colour option for the part	a modifier class: `btn-primary`, `btn-danger`
The instruction booklet	getbootstrap.com/docs
Raw wood and a saw	your own CSS (what you learnt today)

Important: IKEA furniture still needs **you** to know what a table is. Bootstrap is easy **only** if you know CSS. When a Bootstrap page looks wrong, you debug it with the box model, selectors and DevTools, exactly as in note 09.

Words you now own

Term	What it means (plain English)	Example
CSS framework	A ready-made stylesheet with named classes.	Bootstrap, Tailwind (later in the course)
Bootstrap	The most famous CSS framework. We use version 5.3.3, as a local file.	`css/bootstrap.min.css`
`.min.css`	"Minified": all spaces and new lines removed so the file is smaller. Not for reading.	`bootstrap.min.css`
Utility / component class	A class that does one job (`text-center`) or builds one thing (`card`).	`class="btn btn-primary"`
`container`	A centred box with a max width and side padding. (You built this yourself: `max-width` + `margin: 0 auto`.)	`<div class="container">`
`btn` + `btn-primary`	A styled button (base class + colour class). Works on `<button>` and `<a>`.	`<button class="btn btn-primary">`
`card`	A box with border, radius and padding parts (`card-body`, `card-title`, `card-text`).	a dish card
CDN	A public server that hosts files like Bootstrap. Needs internet.	`cdn.jsdelivr.net`
Local copy	The file saved in your project, so it works offline (our class way).	`W2D2/assets/vendor/bootstrap-5.3.3/`

How it works

Step by step: from plain page to Bootstrap

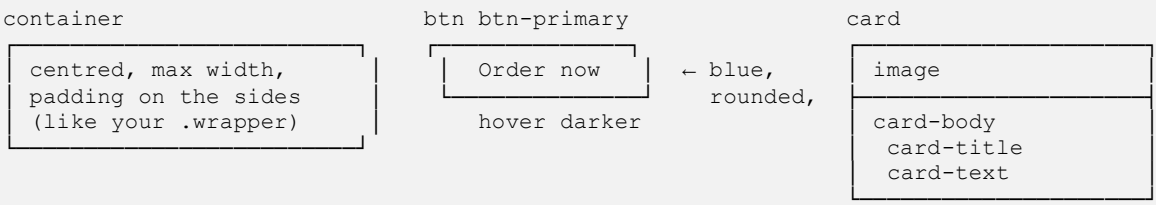
1. Copy `bootstrap.min.css` from `W2D2/assets/vendor/bootstrap-5.3.3/` into your project's `css/` folder.
2. Link it in the `<head>`, **before** your own stylesheet:

```
<link rel="stylesheet" href="css/bootstrap.min.css">
<link rel="stylesheet" href="css/style.css">
```

3. Add Bootstrap's class names to your HTML.

Why before your own file? The cascade (note 04): with equal selectors, the **later** rule wins. Your `style.css` comes later, so your own rules can override Bootstrap's.

The three parts you saw in class



The `btn` class does the shape and padding. The second class (`btn-primary`, `btn-success`, `btn-danger`, `btn-outline-primary`…) picks the colour. You need **both**.

Hand-written CSS vs Bootstrap

	Your own CSS	Bootstrap
Speed	slower to start	very fast to start
Look	unique	many sites look the same ("Bootstrap look")
File size	tiny (a few KB)	big (≈ 230 KB)
Understanding	you know every line	you must learn its class names, and still need CSS to fix things
Best for	learning, small unique sites	admin panels, quick prototypes, team projects

Interviewers in Kathmandu often ask: "Have you used Bootstrap? Can you also write the CSS without it?" The right answer is **yes to both**.

Code you built today

Step 1: the plain page (raw wood, no CSS yet)

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House | Plain</title>
</head>
<body>
  <div>
    <h1>Momo House</h1>
    <p>Fresh momo in Thamel, made by Didi.</p>
    <button type="button">Order now</button>
    <div>
      <img src="images/momo-plate.svg" alt="A plate of steam momo with chutney" width="300">
      <h2>Steam momo</h2>
      <p>Ten pieces, Rs. 150.</p>
    </div>
  </div>
</body>
</html>
```

Predict: with no CSS at all, where on the screen will the heading start? Will the button look like a "modern" button?

Step 2: the same page with Bootstrap (IKEA parts)

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House | Bootstrap</title>
  <link rel="stylesheet" href="css/bootstrap.min.css">
</head>
<body>
  <div class="container">
    <h1>Momo House</h1>
    <p>Fresh momo in Thamel, made by Didi.</p>
    <button type="button" class="btn btn-primary">Order now</button>
    <div class="card">
      <img src="images/momo-plate.svg" class="card-img-top" alt="A plate of steam momo with chutney">
      <div class="card-body">
        <h2 class="card-title">Steam momo</h2>
        <p class="card-text">Ten pieces, Rs. 150.</p>
        <a href="index.html" class="btn btn-outline-primary">See the menu</a>
      </div>
    </div>
  </div>
</body>
</html>
```

- `container` centres the content and adds side padding.
- `btn btn-primary` gives a blue, rounded button with a hover effect.
- `card` + `card-img-top` + `card-body` + `card-title` + `card-text` build the dish card.
- We did not write one line of CSS. Bootstrap's file did it.

Predict: make the window narrow, then wide. What does `container` do to the width of the content?

Step 3: your own rules on top

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Momo House | Bootstrap + my CSS</title>
  <link rel="stylesheet" href="css/bootstrap.min.css">
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <div class="container">
    <h1>Momo House</h1>
    <div class="card dish-card">
      <div class="card-body">
        <h2 class="card-title">Jhol momo</h2>
        <p class="card-text">In warm sesame soup, Rs. 200.</p>
        <button type="button" class="btn btn-primary">Add to order</button>
      </div>
    </div>
  </div>
</body>
</html>
```

File: `css/style.css`

```
/* my own rules come AFTER Bootstrap, so they can override it */
h1 {
  color: #990011;
}

.dish-card {
  max-width: 320px;
  border: 2px solid #14213D;
}
```

- Your `style.css` is linked **after** Bootstrap, so your `h1` colour wins.
- `.dish-card` adds your own width and border to Bootstrap's card.

Predict: swap the two `<link>` lines (your file first, Bootstrap second). Does the h1 stay cherry? Check in F12 → Styles which rule is crossed out.

Quick reference

Want	Bootstrap class	You built it yourself as
Centred page width	`container`	`max-width: 960px; margin: 0 auto; padding: 0 16px;`
A button	`btn btn-primary`	padding + background + border-radius + `:hover`
An outline button	`btn btn-outline-primary`	border + colour, no background
A card	`card` › `card-body` › `card-title`, `card-text`	`.card { padding; border; border-radius; }`
Centre text	`text-center`	`text-align: center;`
Space below	`mb-3`	`margin-bottom: 1rem;`

Mistakes everyone makes

You see…	Because…	Fix	Planted in
No Bootstrap styles at all	The `<link>` path is wrong, or the file was not copied into `css/`	Check Network for 404; copy the file	D2-FF6
The button is plain grey text	Only `btn-primary`, no `btn` (or only `btn`)	Use both: `btn btn-primary`	D2-FF6
Your own colour never shows	Your stylesheet is linked **before** Bootstrap	Link yours **after**	(the cascade, note 04)
No styles when the Wi-Fi is off	Linked to a CDN URL	Use the local copy	(class setup)
A Bootstrap page breaks and you	You don't know the CSS under it	Inspect: it is the same box	(notes 05–09)

You see…	Because…	Fix	Planted in
don't know why		model and selectors	

Check yourself

1. What is a CSS framework? Name one advantage and one disadvantage.
2. Why do we link `bootstrap.min.css` before our own `style.css`?
3. What do `btn` and `btn-primary` each do? Why do you need both?
4. Which of your own CSS rules from today does `container` replace?
5. Why do we use a local copy of Bootstrap in class instead of a CDN link?
6. An interviewer asks: "Why learn CSS if Bootstrap exists?" What do you say?
7. Explain Bootstrap to a friend using IKEA furniture and raw wood.

Try it

- **D2-FF6** Bootstrap try-it: add `container`, `btn btn-primary` and a `card` to a plain page with the local Bootstrap file; compare with your hand-written CSS from D2-13.

Coming later

Session **3.3 Bootstrap deep dive**: the grid system (rows and columns), navbars, forms and responsive breakpoints. Bootstrap's grid uses flexbox, which you meet in session 3.2. Tailwind (another framework) comes later in the program.

AI rule for this week

Stage 1: AI as Tutor. AI may explain; you type every line. A good prompt: *"Explain what the Bootstrap classes container, btn, btn-primary and card each do in plain CSS terms (padding, margin, border). Don't build a page for me."*

Go deeper

- Bootstrap docs (Wi-Fi): getbootstrap.com/docs/5.3 → Components → Buttons, Card; Layout → Containers.
- MDN: CSS frameworks and libraries (https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Client-side_tools/Overview)

---

*Remember the One Rule: **You may only ship code you can explain, line by line.*** *Next note: 11 · Day 2 cheatsheet.*

11 · Day 2 Cheatsheet: Semantic HTML, A11y, Media → CSS Basics

File: notes/11-day2-cheatsheet.md

TSI Nepal · AI-Powered Full Stack Developer Program · Week 2 · Day 2 · Keep this next to your keyboard **Layer:** Frontend

The day in one sentence: name the boxes (semantic HTML), make them work for everyone (a11y), then dress them with one shared stylesheet (CSS).

The analogies (same words as class)

Topic	Picture
CSS idea	**Skeleton & outfit**: same HTML, new stylesheet
Semantic HTML	**Boxes / rooms with names**: kitchen, bedroom, not "room 1, room 2"
Accessibility + meta	**The parcel label read aloud**
Audio, video, iframe	**TV, radio, and a window into another room**
3 ways to add CSS	**Sticky note** (inline) / **outfit list** (internal) / **school uniform** (external)
Selectors	**Calling students in class**: everyone / red team / Aarav by name / red team in row 2
Colours, units	**Paint shop** / **measuring tape**
Box model	**Parcels being shipped**: item / bubble wrap / cardboard / gap
Display	**The queue vs the bench**
Bootstrap	**IKEA for websites**

A complete page that uses (almost) everything

File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Momo House in Thamel: steam, fried and jhol momo made by Didi.">
  <title>Momo House | Home</title>
  <link rel="icon" href="images/favicon.svg" type="image/svg+xml">
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header class="site-header">
    <a href="index.html"><img src="images/logo-momo-house.svg" alt="Momo House home page" width="60"></a>
    <nav>
      <ul class="menu">
        <li><a href="index.html">Home</a></li>
        <li><a href="index.html">Menu</a></li>
        <li><a href="index.html">Order</a></li>
      </ul>
    </nav>
  </header>
  <main class="wrapper">
    <h1>Fresh momo in Thamel</h1>
    <section>
      <h2>Today's specials</h2>
      <article class="card">
        <h3>Jhol momo</h3>
        <p class="price">Rs. 200</p>
      </article>
    </section>
    <figure>
      <video controls width="320" poster="images/poster-momo-steam.svg">
        <source src="media/momo-steam.mp4" type="video/mp4">
        Your browser cannot play this video.
      </video>
      <figcaption>Didi's steamer at work.</figcaption>
    </figure>
  </main>
  <footer class="site-footer">
    <p>&copy; 2026 Momo House, Thamel, Kathmandu</p>
  </footer>
</body>
```

```
</html>
```

File: `css/style.css`

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, Helvetica, sans-serif;
  line-height: 1.6;
  color: #14213D;
  background-color: #fdf6ec;
}

.site-header,
.site-footer {
  padding: 12px 24px;
  background-color: #14213D;
  color: white;
}

.menu {
  list-style: none;
  margin: 0;
  padding: 0;
}

.menu li {
  display: inline-block;
}

.menu a {
  display: inline-block;
  padding: 8px 12px;
  color: white;
  text-decoration: none;
}

.menu a:hover {
  background-color: #990011;
}

.wrapper {
  max-width: 960px;
  margin: 0 auto;
  padding: 24px;
}

.card {
  width: 300px;
  padding: 16px;
  border: 2px solid #14213D;
  border-radius: 12px;
  background-color: white;
}

.price {
  font-size: 1.25rem;
  font-weight: bold;
  color: #990011;
}
```

Semantic tags

Tag	Use for	Rule
`header`	logo, site name, nav	top of page or article
`nav`	main links	links only
`main`	this page's main content	**one** per page

Tag	Use for	Rule
`section`	themed group	starts with a heading
`article`	stands alone (post, card)	makes sense elsewhere
`aside`	related, extra	not the main story
`footer`	contact, ©	bottom
`figure` + `figcaption`	image + caption	
`button` / `a href`	**do** something / **go** somewhere	never `div class="btn"`

Accessibility + head checklist

`<html lang="en">` · a real `<title>` · `meta charset` · `meta viewport` · `meta description` · favicon · every `img` has `alt` (decorative: `alt=""`) · link text makes sense alone (no "click here") · every input has `<label for>` + `id` · headings in order h1 → h2 → h3 · strong contrast · the **Tab walk** reaches everything · **Lighthouse** → Accessibility.

Media

Tag	Must have
`<audio>`	`controls`, correct `src`
`<video>`	`controls`, `<source>` **inside** with the right `type`, `poster` path right
`<iframe>`	`src`, `title`, `width`, `height`

Boolean attributes (`controls`, `muted`, `loop`, `autoplay`) are ON when present: `controls="false"` is still ON. Autoplay needs `muted` or a file with no sound track (and is best avoided).

CSS rule and linking

```
selector { property: value; property: value; }
<link rel="stylesheet" href="css/style.css">     ← in <head>, spelt exactly
/* comment */                                     ← never //
```

Cascade: equal selectors → **last wins**. Stronger selector → **wins anyway**: `#id` > `.class` > `element`.

Selectors

Write	Means
`p`	every p
`.veg`	class veg
`#cart`	id cart
`h1, h2`	both (no trailing comma!)
`nav a`	a inside nav
`.dish.spicy`	one element, both classes
`a:hover`	mouse over a

Names are case-sensitive. HTML classes are separated by **spaces**, never commas.

Values

Kind	Examples	Watch out
Colour	`maroon` · `#14213D` · `#fff` · `rgb(20, 33, 61)` · `rgba(0, 0, 0, 0.5)`	hex = 0–9 a–f, 3 or 6 digits; rgb = 3 numbers
Length	`16px` · `50%` · `1.5rem` · `2em` · `0`	unit glued on, no space; no unit only for 0
Font stack	`"Momo Sans", Arial, Helvetica, sans-serif`	always end with a generic family

Kind	Examples	Watch out
Line height	`1.6`	no unit

Box model

```
margin (gap) › border (cardboard) › padding (bubble wrap) › content (item)
visible width = border + padding + width + padding + border     ← unless box-sizing: border-box
padding / margin shorthand: top right bottom left (TRouBLe) · 1, 2, 3 or 4 values
border: 2px solid #14213D;   ← width STYLE colour (no style = no border)
centre a box: max-width: 960px; margin: 0 auto;
circle: equal width + height, border-radius: 50%
```

Display

Value	Behaves
`block`	own line, full width (div, p, h1, li, section…)
`inline`	in the text line; ignores width, height, vertical margin (a, span, strong)
`inline-block`	in the line, but sizes and margins work (nav links, badges)
`none`	gone, no space
`visibility: hidden`	invisible, space kept

Horizontal nav = `ul { list-style: none; padding: 0; }` + `li { display: inline-block; }` + `a { display: inline-block; padding: …; }`

Debugging CSS in 4 questions

1. **Loaded?** Network → `style.css` 200 (not 404); `rel="stylesheet"`; right path.
2. **Matches?** Inspect → is the rule listed in Styles? Compare names letter by letter.
3. **Understood?** A yellow ⚠ = invalid value or unknown property → dropped.
4. **Won?** Crossed out with no ⚠ = a later or stronger rule won.

What the browser drops: a bad declaration → that line only · a bad selector (`h1, h2,` · `a:hoverr` · after `//`) → the whole rule · a missing `;` → two declarations · a missing `}` → the next rule is swallowed · a bad `<link>` → everything.

Keyboard and DevTools

Do	Keys / place
Inspect an element	right-click → Inspect, or Ctrl+Shift+C
See box sizes	Elements → Computed → box diagram
See dropped/losing CSS	Elements → Styles (⚠ / crossed out)
See missing files	Network → refresh → red 404
Accessibility report	Lighthouse → Accessibility → Analyze
Keyboard test	Tab / Shift+Tab / Enter / Space
Rename a tag pair	just retype the opening tag (linked editing is on in `W2D2`)
HTMLHint problems	Ctrl+Shift+M

Coming later

Flexbox and grid (3.2) · positioning (3.2) · responsive design and media queries (3.2) · the Bootstrap grid (3.3) · JavaScript for buttons (4.2).

AI rule for this week

Stage 1: AI as Tutor. AI explains; you type every line. Debug first with the 4 questions; then ask. A good prompt: *"Here is one CSS rule that DevTools shows crossed out without a warning triangle. Explain why it lost. Don't rewrite my stylesheet."*

---

*Remember the One Rule: **You may only ship code you can explain, line by line.*** *Next session: 3 / 3.2: CSS from basics to layout (flexbox and grid). Homework first:* `homework/W2D2-Homework-Sheet.md`.